Contents

Intelligent Guide — Requirements 2
1. Problem & Context 2
2. North Star & Non-Goals 2
3. Design Model — Three-Phase Guide 4
   3.5 Phase 0 — Visitor to Registered User (the pre-Guide phase) 4
4. The Journey — 7 Steps 6
5. Required New Features 7
   5.1 Demo Table macro (Hook step 2) 8
   5.2 Public Spar endpoint (Curriculum step 5) 9
   5.3 Quick Bot wizard (Curriculum step 3, friction-reduction) 9
   5.4 Curriculum Cup — template + clone (Curriculum step 6) 10
   5.5 Coaching card (Curriculum step 7) 11
   5.6 Journey schema migration 12
   5.7 Discovery rewards — archetype-activating micro-rewards 12
   5.8 Rookie Cup — second-tournament template + clone (Specialize Competitor rec #1) 13
   5.9 Deterministic bracket seeding (new tournament infrastructure) 14
6. Specialize Phase — Four Buckets 14
   6.1 Archetype detection 14
   6.2 Designer bucket 15
   6.3 Trainer bucket 16
   6.4 Competitor bucket 16
   6.5 Explorer bucket 16
   6.6 Why four buckets (and not three or five) 17
7. Scoring & Surfacing 17
   7.1 Card surfacing rules 17
   7.2 Ordering within a bucket 17
   7.3 Dismissal and re-engagement 18
   7.4 Stagnation handling — decay and exhaustion 18
   7.5 Progressive intensity 18
8. Backend Architecture 19
   8.1 New service files 19
   8.2 Updated service files 19
   8.3 New endpoints 19
   8.4 Schema additions 19
   8.5 Server-authoritative, client-rendering 22
9. UI Changes 22
   9.1 `JourneyCard` rewrite 22
   9.2 `GuideOrb` retained 22
   9.3 `SlotGrid` — locked during journey, unlocked at graduation 22
   9.4 Reward celebrations 23
   9.5 Admin experience 23
10. Testing 24
   10.1 Backend regression / unit tests 24
   10.2 UI / component tests (landing) 25
   10.3 E2E tests — added to the QA suite 25
   10.4 Fixtures & helpers 26
   10.5 What we actually care about (coverage priorities) 26
   10.6 QA/Dev CLI (`um`) enhancements 26
11. Rollout Plan 28
   Phase 0 — Visitor → Registered User (ships first; everything else depends on signups) 28
   Phase 1 — Hook + Curriculum foundation 28
   Phase 2 — Specialize phase 28
   Phase 3 — Re-engagement & polish 28
   Phase 4 — Measurement 29
12. Decisions Captured 29

13. Open Questions — For Implementation Review . . . 33
13.1 Rewards & economy . . . 34
13.2 Curriculum Cup specifics . . . 34
13.3 Specialize phase . . . 34
13.4 Feature details . . . 35
13.5 Resolved during requirements (for the record) . . . 35
14. Success Criteria . . . 36
Appendix A — Glossary . . . 36
Appendix B — File Touch List . . . 36

Intelligent Guide — Requirements

Status: Draft v1 — pre-implementation **Author:** Joe Pruskowski (with Claude) **Date:** 2026-04-24 **Supersedes:** the current fixed 7-step onboarding journey (see `backend/src/services/journeyService.js` and `landing/src/store/guideStore.js`)

---

1. Problem & Context

The platform is pre-launch with a first-cut Guide: a 7-step linear onboarding journey followed by a user-configured 8-slot shortcut tray. Today's Guide:

- Fires steps on a mix of server events (e.g. first training run) and client triggers (page visits, popup dismissal)
- Has **no behavior after step 7** — once onboarded, the Guide stops guiding
- Does not adapt to user activity, answer questions, or nudge toward underutilized features
- Mixes onboarding moves (FAQ visit, tournament popup) with high-commitment moves (first training run) in the same linear sequence, making the funnel fragile

Since there is no live user data yet, we have **total freedom to redesign** without migration concerns. This document specifies a replacement.

2. North Star & Non-Goals

North Star metric

> **% of registered users whose bot has played at least one tournament match within 30 days of signup.**

This captures the unique value of the platform — users bringing bots they built/trained into competitive play. It subsumes most secondary goals: a user at North Star has gone through create, train, and compete.

Secondary metrics

- Median time from signup → first bot created (Curriculum funnel speed)
- Median time from signup → first tournament entered (Curriculum completion)
- % of users who enter a second tournament within 14 days of their first (retention)
- Guide recommendation click-through rate (Specialize phase quality)

Measurement — Dashboard & Time-Series Persistence (v1 requirement)

Metrics without a view into them are just hope. The dashboard and time-series store are part of v1, not a later add-on:

- **Admin dashboard** at `/admin/guide-metrics` (or similar) renders:
  - North Star metric with its 30-day trend line
  - Each secondary metric with current value + trend
  - **Funnel visualization** — % of users currently at each of the 7 journey steps, with drop-off per step
  - **Specialize-phase metrics** — bucket distribution (what % of users are Designer-dominant, Trainer-dominant, etc.), **per-bucket archetype score histograms** (not just dominant assignments — the full distribution of individual scores, so we can detect miscalibrated normalization constants per §6.1), recommendation click-through rate per bucket, dismissal rate, card-refresh rate

  - **SlotGrid customization metrics** — % of graduates who edit their default tiles within 7 days, which tiles are added/removed most often, whether edit patterns cluster by archetype (feeds the deferred archetype-seeded-defaults decision — see §13 Q12)
  - **Phase 0 conversion funnel** (§3.5.5) — landing conversion rate (% visitors → signup within 7 days); guest progression funnel (land → play PvAI → watch demo → click "Build a bot" → signup); time-to-signup median; Hook-credited-at-signup rate (signal that guest mode is working); signup method split (OAuth vs email)
  - **Cohort slicer with admin-selectable granularity** (Day / Week / Month, default Week) — filter all of the above by signup bucket, so we can compare whether users who signed up in bucket N behave differently from bucket N+1 (detects Guide-change regressions or wins). Granularity is a UI-level view pivot (same query with a different `DATE_TRUNC` — no separate aggregation per granularity), picked by the admin per-view rather than hardcoded. Admins can switch based on signup volume: daily for high-volume periods where fine-grained signal is needed; monthly when signup volume is too low for smaller buckets to be meaningful.
- **Time-series persistence**:
  - Raw events already carry timestamps (`bot.createdAt`, `tournament.completedAt`, training run timestamps, etc.) — the dashboard queries these directly for event-level metrics
  - **Aggregate snapshots** stored in a new `metricsSnapshot` table with shape `{ id, date, metric, value, dimensions (JSON) }`, written by a daily cron job at UTC midnight
  - Retention: indefinite. Aggregates are small (a few KB/day); long-term trend visibility compounds as the user base grows
- **Why v1, not later**: without the dashboard we cannot tell whether the Guide is working. Without time-series data we cannot tell whether a change moved the needle. Adding snapshotting later requires backfilling from raw events, which is painful for derived metrics (e.g. bucket distribution) and often imprecise.

Preventing internal-usage pollution

Metrics are meaningless if admin dogfooding, QA test runs, and developer smoke testing count as "real users." The Guide actively defends against this with five layers:

1. **Environmental isolation.** Dashboards query the **production database only**. Staging and dev have separate databases; activity there is structurally invisible to production metrics. (Restated here because it's the first line of defense — not a new requirement.)
2. **`isTestUser` flag on every User row.** Default `false` for real users; defaulted to `true` on creation for: (a) any user with an admin role, (b) accounts created via `seed.js` / `setup-qa-users.sh`, (c) accounts with email domains matching the `metrics.internalEmailDomains` SystemConfig list (§8.4).
3. **Uniform metrics filter.** All aggregations (North Star, funnel, bucket distribution, archetype histograms, cohort analysis, daily `metricsSnapshot` cron) filter `WHERE user.isTestUser = false` at the aggregation layer. Single source of truth — no per-query risk of forgetting the filter.
4. **Admin opt-in to count.** Settings page includes *"Include my activity in platform dashboards"* — default **off** for admins. When on, the admin's `isTestUser` flips to `false` and their activity contributes to metrics. Most admins leave it off (their dogfooding shouldn't represent the real-user experience); admins with strong personal usage patterns who want their signal counted can opt in.
5. **User-facing behavior unchanged.** Internal users still earn TC, progress through journey steps, complete recommendations, dismiss cards — the full Guide experience works. Only the *aggregate metrics* exclude them.

Dashboard transparency: a footer note on every metrics view displays *"excluding N test users"*. A sudden jump in this count is itself a signal (e.g. a CI bug creating accounts without the flag would show up here).

Admin overrides: the admin user-list page has a per-user `isTestUser` toggle for manual adjustment — supports the rare case where a test user becomes real, or vice versa.

Non-goals for v1

- Free-form Q&A / conversational AI coach — future phase
- Content authoring UI for rules/catalog — config files only in v1
- Cross-device state sync beyond what already exists
- Localization — English only in v1
- Real-time streaming metrics (per-second updates) — daily snapshots are enough for v1; upgrade to hourly or event-driven later if needed
- Public-facing metrics (user-visible leaderboard of the platform's own engagement) — admin-only in v1

3. Design Model — Three-Phase Guide

The Guide walks every user through three distinct phases, each with different goals, UI, and recommender logic.

Phase	Goal	Duration	UI	Recommender
Hook	"Oh, I get what this platform is"	First session, ~5 min	Single prominent JourneyCard	Linear, 2 fixed steps
Curriculum	Walk them end-to-end through the bot lifecycle	First week-ish	Single JourneyCard, step-by-step	Linear, 5 fixed steps
Specialize	Surface depth in the phase they care about most	Ongoing, forever	Up to 3 cards at a time	Faceted, archetype-weighted

Why three phases?

Earlier iterations of this discussion proposed two phases (sampling → specializing). We split sampling into two because they have different jobs:

- **Hook** is pure engagement — minimum friction, immediate "this is cool" moment, user invests nothing
- **Curriculum** is structured learning — requires the user to commit (create a bot, run training) and builds toward a payoff

Collapsing them leads to the problem the current journey has: mixing low-commitment moves (visit FAQ) with high-commitment moves (first training run), causing drop-off in the middle.

Progression

- Every user starts in Hook
- Hook → Curriculum when both Hook steps complete
- Curriculum → Specialize when all Curriculum steps complete (user has a bot that has competed)
- Specialize is an absorbing state — the user stays here unless they manually restart onboarding

3.5 Phase 0 — Visitor to Registered User (the pre-Guide phase)

Before the Guide can do any work, a visitor has to become a registered user. **This is the highest-leverage conversion point on the platform** — everything else we've designed only matters if visitors successfully sign up. Phase 0 defines the pre-signup experience.

Why this is a dedicated phase, not "onboarding work"

Landing page + signup is where industry conversion rates are lowest (typically 2–5% of visitors become registered users). A popup-to-register after 30 seconds of PvAI interrupts flow at the worst moment. The Guide we designed for registered users (Hook → Curriculum → Specialize) is useless if it never sees a registered user. Phase 0 is the "Phase before Hook."

Today's flow (baseline)

The current homepage offers two CTAs: "View Tournaments" and "Play XO" (the latter only for logged-out visitors). Clicking "Play XO" drops the visitor into `/play?action=vs-community-bot` — a PvAI match against a community bot. A signup popup eventually interrupts the game. Conversion friction compounds:

- Hero text is generic ("Classic games. Trainable AI. Real-time multiplayer tournaments.") — doesn't surface the unique value prop (*users build bots that compete*)
- "Play XO" is the least-unique thing the platform does — the unique thing (bot-vs-bot, tournaments, training) is invisible on the landing page
- Popup interrupts flow mid-game, out of context
- Signup requires email verification before platform access — momentum dies while the verification email lands

Phase 0 replaces this with **progressive engagement + contextual ask**.

3.5.1 Landing page — live demo hero Replace the static text hero with an **embedded live bot-vs-bot match** running in real time at the top of the homepage. Reuses the §5.1 Demo Table macro infrastructure — same allowlisted pairings, same rendering surface. Cycles to a new match automatically every ~30 seconds so repeat visitors see variety.

Next to the live demo, three CTAs in a progressive engagement ladder:

CTA	Commitment	What it does	Conversion role
"Watch another match"	Zero	Restarts the demo with a new bot pairing	Keeps visitor watching — pure demonstration
"Play against a bot"	Low	Guest PvAI against a community bot (existing flow)	Low-friction hook — lets them touch the platform
"Build your own bot"	Medium — triggers signup	Click opens signup modal with contextualized copy	**The conversion CTA** — the platform's hero feature is gated behind this

Why this ladder works: the visitor sees the unique value prop (bots playing each other) *before* any click. The "Build your own bot" CTA explicitly surfaces the feature they can't access without an account — making the signup ask motivated by a feature they want, rather than an arbitrary popup interruption.

3.5.2 Guest mode (no account, no bot, limited journey) Visitors can use a subset of the platform without an account. Guest identity is **localStorage only** — no DB rows, no cookies requiring consent banners, no privacy complications.

Guest capabilities:

- **YES** Watch live bot battles (the homepage demo)
- **YES** Play PvAI against a community bot (existing `/play?action=vs-community-bot` flow)
- **NO** Cannot create bots
- **NO** Cannot spar
- **NO** Cannot enter tournaments
- **NO** Cannot build journey progress past Hook step 2

Journey events for guests are recorded in localStorage under `guideGuestJourney`:

```
// localStorage: guideGuestJourney
{
  hookStep1CompletedAt: '2026-04-24T10:15:22.314Z',  // first PvAI game completed
  hookStep2CompletedAt: '2026-04-24T10:18:05.812Z',  // bot-battle demo watched ≥ 2 min
}
```

Only Hook steps 1 and 2 are eligible — Curriculum steps require signup by design (they involve creating/training/competing with a user-owned bot).

Session expiry: guest state persists indefinitely in localStorage. No 30-minute timer (keeps it simple). Visitor can return days later and their "already watched a demo" state is still there, crediting them on eventual signup.

3.5.3 Guest → user transfer on signup When the visitor creates an account, we want them to land **directly in Curriculum step 3 (Create your first bot)** — not back at Hook step 1, which they already did as a guest.

Flow:

1. Client reads `guideGuestJourney` from localStorage on signup submit
2. After successful signup, client posts `POST /api/v1/guide/guest-credit` with the guest events
3. Backend validates and credits steps 1 and 2 on the user's `journeyProgress`
4. Rewards fire as usual (+20 TC for step 2 completion)
5. localStorage `guideGuestJourney` is cleared

Endpoint: **`POST /api/v1/guide/guest-credit`** — authenticated, body: `{ hookStep1CompletedAt, hookStep2CompletedAt }` (both optional).

- Applies the step completions if not already present
- Ignores any other steps (guests can't earn Curriculum credit)
- Idempotent: re-running is a no-op
- Low-risk to trust client data for Hook steps — worst case is a user claims +20 TC without actually watching a demo, which is trivial impact

Contextual signup copy when the user arrives at the signup modal from the "Build your own bot" CTA:

> **Build your first bot** Create a free account to build bots, train them, and enter tournaments. *You've already earned Hook progress — you'll start one step ahead.*

The "you'll start one step ahead" hint only appears if `guideGuestJourney` has entries. Otherwise it's a generic signup ask.

3.5.4 Signup improvements Three targeted changes to `SignInModal.jsx` to reduce friction:

(a) Deferred email verification — do not block platform access

Currently, after signup the user sees a "verify your email" screen and cannot use the platform until they click the verification link. This kills momentum — if the email lands in spam, the user bounces.

Proposed behavior:

- After signup succeeds, the user is **immediately logged in and lands in Curriculum step 3** (Quick Bot wizard)
- A soft banner across the top of the app reads: *"Verify your email to enter tournaments — [resend link]"*
- The user can fully use the platform (play, create bots, spar, dismiss recommendations, earn discovery rewards) — only tournament entry is gated behind verified email
- When they want to enter a tournament (Curriculum step 6 or any Specialize tournament), a modal prompts them to verify; the verification email is resent at that moment

This replaces "verify to unlock everything" with "verify to unlock the specific thing you want." Preserves momentum; the user verifies when they have a concrete reason to.

Spam/bot protection stays intact: existing honeypot + 3-second timing check + OAuth still apply; only the *post-signup blocking behavior* changes.

(b) Contextual signup copy — when the modal opens from the "Build your own bot" CTA, copy names the feature they're unlocking (see §3.5.3). For other entry points, copy stays generic.

(c) OAuth prominence — no change. Current layout (OAuth buttons alongside email/password) is kept as-is.

Deferred to v2 (explicitly out of scope for Phase 0):

- **Magic-link signin** (passwordless email-link signin) — nice to have, requires solid email deliverability, current flow works
- **Social-proof counters on the landing page** — "23 tournaments running now," etc. Risky to ship with low engagement numbers; revisit once the platform has real volume

3.5.5 Measurement for Phase 0 Add to the §2 dashboard:

- **Landing conversion rate** — % of unique visitors who sign up within 7 days of first visit
- **Guest progression funnel** — % of visitors who: land → play PvAI → watch demo → click "Build your own bot" → complete signup
- **Time-to-signup** — median time from first landing visit to signup (separately for each CTA path)
- **Hook-credited-at-signup rate** — % of new signups who arrive with Hook steps already credited from guest activity (signal that the guest mode is working)
- **Signup method split** — OAuth vs email/password

These are the Phase 0 funnel; without them we can't tell whether the new model converts better than the old.

3.5.6 What this doesn't cover Phase 0 is specifically about **visitor → registered user**. It does not cover:

- Account recovery / password reset (existing flow stays)
- Profile setup / display name selection (existing flow; could be simplified in a future pass)
- Social graph / invite flow (not yet a platform feature)
- Marketing surfaces (ads, SEO, referral links) — outside the Guide's scope

Once a visitor signs up and lands in Curriculum step 3, Phase 0 is done; the rest of the Guide (§4–§9) takes over.

4. The Journey — 7 Steps

All triggers are **server-detectable**. This is a deliberate change from the current journey, which has three client-triggered steps (2, 4, 7). Server-detection eliminates state drift and removes the need for the `POST /journey/step` endpoint.

#	Phase	Step	Trigger	Reward	Depends on
1	Hook	Play your first PvAI game	`game.completedAt != null` where user is human player	—	—
2	Hook	Watch two bots battle	Demo table spectated ≥ 2 min OR to completion	**+20 TC**	Demo Table macro (§5)
3	Curriculum	Create your first bot	`bot.userId == currentUser && bot.createdAt != null`	—	—
4	Curriculum	Train your bot	First completed training run (`mlService` event)	—	—
5	Curriculum	Spar — your bot plays a casual match	`botGame.tournament~~Id~~ == null` AND user owns one of the bots		Public Spar endpoint (§5)
6	Curriculum	Enter a tournament with your bot	`tournamentParticip~~ant~~.botId` belongs to user		Beginner-tournament matchmaking (§5)
7	Curriculum	See your bot's first tournament result	Tournament reaches `COMPLETED` AND user's bot has `finalPosition`	**+50 TC**	Coaching card (§5)

Reasoning per step:

- **Step 1** — PvAI is the lowest-friction action; gets user to "I played a game" fast. No reward because the reward is enjoying the game itself.
- **Step 2** — Bot-watching is the *unique* thing this platform offers. Showing it early (before asking the user to build anything) anchors the value proposition. Small reward to introduce the credit system.
- **Step 3** — First asks the user to produce something. Friction matters here — see Quick Bot wizard in §5.
- **Step 4** — Ties the bot to a model. Many platforms stop at "create" but training is what makes bots *improve*.
- **Step 5** — Spar bridges training (abstract ML metrics) to competition (real match outcome). Low stakes, quick feedback. Without Spar, users can't test training before committing to a tournament.
- **Step 6** — The first public match. This is where the platform's social/competitive loop begins.
- **Step 7** — Closes the loop: "I built a thing and it did something in the world." The +50 TC reward is meaningful — TC has a 5× weight in the tier-progression activity score, so +50 TC = +250 activity-score points, a substantial chunk toward the user's next tier. (TC is not a spendable currency in the current credit system; it's an *earned progression metric*, see `creditService.js`.)

Graduation from Curriculum to Specialize happens at step 7 completion. At that moment the user sees a "Welcome to Specialize" state — the JourneyCard transforms into a "What's next for you" card stack.

Restart behavior: "Restart onboarding" (already exists as `POST /guide/journey/restart`) resets all 7 steps and drops the user back into Hook. Specialize recommendations still render based on activity history — a user who restarts the journey but has 20 tournaments played still gets Competitor recommendations.

5. Required New Features

"Programming is not a constraint" — this journey requires several small but real features. Ordered by dependency.

5.1 Demo Table macro (Hook step 2)

`POST /api/v1/tables/demo` — authenticated, returns `{ tableId, botA, botB }`.

- Picks a system-bot pairing at random from a **curated allowlist** of pre-verified matchups (see below)
- Creates a table seated with both bots, flagged `isDemo = true` and scoped private to the creator
- Starts the match
- Returns a table id; client redirects user to `/tables/:id` in spectator mode

Bot pairing: curated-random allowlist

Pairings live in `tournament/src/config/demoTableMatchups.js` (plain JS, editable via PR — not SystemConfig; this is a brand/quality decision, not a tunable knob). Each entry is a pair of `botModelId` strings known to produce watchable demo matches. Demo endpoint picks uniformly at random from the list.

Pairing	Expected character	Why it's in the allowlist
Copper vs. Sterling	Sterling usually wins, but ≥ 7 moves with visible strategy; occasional Copper upsets	The default "strong demo" — reliably interesting
Rusty vs. Copper	Copper wins most; Rusty lucky-wins occasionally	Shows that even a novice bot has *some* chance — demystifies the tiers
Copper vs. Copper	Same-tier blocking dance, draws common	Demonstrates the "both bots block threats" strategic layer
Sterling vs. Sterling	Frequent draws (XO is a solved game at near-perfect play)	Shows that XO at high play is often a stalemate — a real game-theory insight

Why curated-random beats both extremes:

- vs. **single static pair**: variety starts on day one — a user who watches 3 demos in a session sees different matchups, signaling platform range. No extra work — the allowlist is one more level of indirection over static.
- vs. **uncurated random**: the "broken bot accidentally in a demo" risk is small but asymmetric — a bad Hook-phase first impression is disproportionately costly. Allowlist guarantees every eligible pairing produces watchable games.

Extending is a PR adding one line to `demoTableMatchups.js`. When Magnus or future tiers get well-tested matchups (e.g. "Magnus vs. Sterling, always a draw"), they join the list.

Why not reuse existing bot-game endpoints? `POST /api/v1/bot-games` requires ADMIN/BOT_ADMIN. We need a user-scoped variant. The demo path is simpler than a generic user-to-user bot match because the bots are pre-selected by the platform.

No rate limit — garbage-collected instead. A rate limit punishes legitimate use cases (re-watching different demos back-to-back) without solving the underlying concern. GC solves the root cause (clutter) without artificial caps. Three stacked mechanisms:

Mechanism	Fires when	Why
One active demo per user	User creates a new demo while their previous demo is still running — old one is killed and replaced	Prevents accumulation during active use; removes the "I left one running yesterday" problem
Match-complete + 2 min grace	Bot-vs-bot match reaches terminal state → wait 2 min (let user read the result) → delete the table, game row, and associated records	Aggressive cleanup of the common case
Hard TTL of 1 hour	Any demo table older than 1 hour, regardless of state	Safety net for orphans — tab closed mid-match, server restarted, etc.

Plus **demo tables are private to the creator**: the public tables list applies a `where isDemo = false OR createdByUserId = caller.userId` filter. Even if a demo somehow survives GC briefly, it doesn't pollute what other users see.

Implementation note: GC lives in a lightweight sweep (60s interval, like the existing tournament sweep). Expressed as SQL/Prisma predicates against the `Table` + `Game` tables, not a per-demo in-memory timer — survives server restarts.

5.2 Public Spar endpoint (Curriculum step 5)

`POST /api/v1/bot-games/practice` — authenticated, body: `{ myBotId, opponentTier: 'easy' | 'medium' | 'hard' }`.

- Verifies the caller owns `myBotId`
- Picks a system bot matching the requested tier
- Runs a bot-vs-bot match (no tournament id, no match id), flagged `isSpar = true`
- Emits `bot:match:started` so the client can watch

UI: a "Spar now" button on the bot detail page, and a dedicated Guide-delivered card during Curriculum step 5.

No rate-limiting. A user who wants to run 50 practice matches in an hour is engaged, not abusive — that's the healthiest behavior the platform has. Capping that to "force progression" is a crude mechanism that punishes dedication; **progression forcing belongs in the Guide's recommendation logic**, not at the endpoint. The cross-bucket disqualifier in §6.3 ("Trainer whose bot has never played a tournament match → substitute Competitor recommendation") already handles the "stuck in Spar" scenario gracefully.

Retention. Unlike demo tables, spar matches are the user's own work product — they may legitimately want to review a match later ("how did my bot perform against Hard difficulty last week?"). So we preserve rather than aggressively delete:

- **30-day retention** — spar matches are kept for 30 days after completion, then GC'd in the same sweep that handles demo tables
- **Hard TTL of 2 hours for in-flight** — safety net for abandoned spars (server restart, tab closed, etc.)

One active spar per bot — a single bot in two concurrent practice matches is semantically meaningless (the bot can't meaningfully play two games at once from a learning/UX perspective). Starting a new spar for an already-sparring bot kills the previous in-flight match. This is a correctness guard, not a rate limit.

5.3 Quick Bot wizard (Curriculum step 3, friction-reduction)

A 3-click "Quick Bot" creation flow:

1. **Name** (free text, validated)
2. **Persona** (from a curated shortlist of 4–6 personas, each with a one-sentence description)
3. **Confirm** — platform picks the default algorithm/tier

Why: full bot creation has many decisions (algorithm, hyperparameters, persona, etc.). For users in Curriculum, those choices are noise. Quick Bot unblocks step 3 without forcing them through the full form. The full form remains available for users in Specialize-Designer mode who want control.

Default algorithm: minimax, novice tier. Matches the existing system bot `Rusty` (`builtin:minimax:novice`). Plays random valid moves — intentionally weak. This looks like an odd choice at first glance (the user's first bot plays randomly?) but it's deliberate pedagogy: the **first training run (step 4) visibly transforms the bot's behavior** from random to intermediate, making training feel immediately rewarding. If the default were already competent, training would feel pointless.

First-training-run bump: novice → intermediate. After step 4 completes, the user's bot is at the Copper tier (`user:<id>:minimax:intermediate`) — blocks immediate threats, takes immediate wins, otherwise random. Strong enough for Spar (step 5) and competitive in the Curriculum Cup final.

Journey step ordering protects the user from playing a random bot. Steps 5 (Spar) and 6 (Cup) require step 4 to be complete, so the user's bot is always at intermediate or better by the time it faces any opponent.

"Training" is a tier bump, not real ML — and that's fine. For minimax bots, there's nothing to train in the gradient-descent sense; Quick Bot's "first training run" is a difficulty config bump marketed as training completion. Real ML happens in Specialize when a user switches to a non-minimax algorithm (Q-learning, policy gradient, etc.), which is precisely what the §5.7 discovery reward "First bot trained with a non-default algorithm" rewards. So:

- **Minimax = training wheels** (difficulty bumps, no gradient descent)
- **Non-minimax = real ML** (rewarded with +10 TC first time — §5.7)

Implementation note: the step-4 completion trigger needs to fire from EITHER (a) `mlService.js` when a real training run completes, OR (b) the Quick Bot difficulty-bump flow when it completes. Today only (a) fires step 4 — (b) is new.

Admin-configurable via SystemConfig keys:

- `guide.quickBot.defaultTier` = `"novice"` (default) — the tier assigned on bot creation
- `guide.quickBot.firstTrainingTier` = `"intermediate"` (default) — the tier after step 4 completes

These let us tune if post-launch data shows the Cup is too easy or too hard. Keeping them admin-editable also means we can test variants (e.g. default = "intermediate" with first training going to "advanced") without a deploy.

5.4 Curriculum Cup — template + clone (Curriculum step 6)

Rather than dynamically matchmaking against whatever tournaments happen to be running, the Curriculum Cup is a **pre-defined tournament template** that the Guide clones on demand for each user. This gives us a controlled, consistent, fast-to-spawn first-tournament experience with no "hope a suitable tournament exists right now" problem.

This leans on existing platform infrastructure: tournament templates + cloning already exist (`cloneAndSeedPersona` / `syncTemplateSeedsToTournament` in `seedBotService.js`; E2E coverage in `tournament-template-clone.spec.js`). The Curriculum Cup is just a template with a thin Guide-specific clone wrapper.

Master template (seeded once via migration):

- Row in the tournament templates table — display name `"Curriculum Cup"`
- Format: SINGLE_ELIM, best-of-1, 4 slots
- Seeded composition: 1 user placeholder + 3 pre-seeded system bots — **2 Rusty (novice) + 1 Copper (intermediate)**
- Held in `PAUSED` / template-only state — never directly entered; it is cloned, not joined
- No ELO/ranking impact (flag carried by all clones)

No Sterling in the first tournament. A Sterling opponent in a user's first-ever tournament risks a bad first experience if they lose round 1 and never see the flow end-to-end. Sterling is reserved for the second tournament (surfaced via Specialize Competitor bucket, §6.4 — 8-entrant format, Sterling seeded in the opposite bracket arm so the user can't face it until semifinals at earliest).

Themed name pools — hardcoded in `tournament/src/config/curriculumNamePools.js` (plain JS arrays, not SystemConfig — names don't need live tuning and editing is a 30-second PR):

- **Rusty pool (10 names)** — novice tier, brown/autumn/worn themes: > *Rusty Hinge, Old Bolt, Tarnished Penny, Corroded Clasp, Oxidized Ox, Flaking Iron, Patina Pete, Crumbling Cog, Weathered Nail, Dusty Sprocket*
- **Copper pool (8 names)** — intermediate tier, craft/electrical/warm-glow themes: > *Copper Coil, Verdigris, Etched Brass, Burnished Bell, Gleaming Wire, Forge-Marked, Struck Copper, Amber Circuit*
- **Sterling pool (6 names)** — elite tier (reserved for the second-tournament flow), silverware/regal/elegance themes: > *Sterling Monarch, Polished Argentum, Silver Knight, Chromed Crown, Moonlit Blade, The Argent Paladin*
- Picked randomly without duplicates within a single cloned tournament; repeat use across different tournaments is fine.
- Adding or editing names is a source-controlled PR (no live admin tuning needed).

Clone endpoint — `POST /api/v1/tournaments/curriculum-cup/clone` — thin wrapper over the existing template-clone path:

1. Clone the Curriculum Cup template (reuses existing cloning machinery — no new clone logic)
2. Substitute the user's bot into the user placeholder slot
3. Apply `botDisplayNameOverride` per participant from the themed name pools
4. Set flags: `isCup = true`, `createdByUserId = caller`, `visibility = PRIVATE` (same pattern as demo tables — not listed publicly)
5. Transition directly to `IN_PROGRESS` and start round 1
6. Return tournament id; client redirects user to the bracket view

Garbage collection piggybacks on the existing `tournamentSweep` — a new phase: rows with `isCup = true` AND `completedAt < NOW() - 30 days` → soft-delete cascade (tournament + participants + matches + games). Same 30-day window as Spar retention — user may want to re-watch their bracket run.

Why template-clone over on-demand creation:

Aspect	On-demand creation	Template + clone
Speed	Slower (DB setup per entry)	Faster (rows are templated, not computed)
Consistency	Logic drifts over time	Master template is the source of truth

Aspect	On-demand creation	Template + clone
Testability	Test create logic	Test the template once, clone once, done
Changeability	Code change + deploy	Edit the template row; next clone reflects it
Admin override	Harder	Admin tunes the Cup without code changes
Reuse existing code	No — new path	Yes — existing tournament-template infrastructure

Flags introduced

- `isCup` (new) — distinct from `isTest` (e2e-hidden) and `isDemo` (tables, not tournaments). Used for: private visibility, 30-day GC, ELO exclusion, optional "Curriculum Cup" badge in the user's tournament history.

5.5 Coaching card (Curriculum step 7)

When the user's first Curriculum Cup completes, the Guide shows a **one-shot result-interpretation card** with a concrete "what to try next" CTA. This is a *teaching* affordance — it exists because a fresh Curriculum graduate hasn't yet learned to interpret a tournament outcome on their own. The coaching card appears on Curriculum step 7 only; Specialize-phase tournament results do not trigger coaching (§6's recommendation system handles post-Curriculum guidance).

4-branch decision tree

Inputs available at step 7:

- `finalPosition` (1st / 2nd / 3rd / 4th — with 3rd and 4th representing the first-round losers in a 4-bracket)
- `trainingRuns` count for the entered bot
- Minor inputs for future refinement but not used in v1: average game length, rounds won vs. lost, bot algorithm

Four branches, first match wins:

#	Match condition	Title	Body	CTA
1	`finalPosition == 1`	**[1st]** Champion! Your bot won the Curriculum Cup.	Your bot is ready for a bigger test. The Rookie Cup has 8 opponents and includes Sterling — a bot that plays to win.	**Enter Rookie Cup** → clones a fresh Rookie Cup (§5.8)
2	`finalPosition == 2` (runner-up — lost in the final)	**[2nd]** Runner-up! You made it to the final match.	So close. A second training run could push your bot over the edge.	**Train again** → opens the training UI for this bot, fires step-4-repeat
3	`finalPosition ∈ {3, 4}` AND `trainingRuns == 1`	Tough first outing.	Most bots need more practice to win their first tournament. Try another training session — your bot can get stronger.	**Train again** → opens the training UI
4	`finalPosition ∈ {3, 4}` AND `trainingRuns >= 2`	Tough draw.	Even well-trained bots can lose their first tournament. Try a different algorithm — Quick Bot uses minimax, but Q-learning or policy gradient might suit your bot better.	**Switch algorithm** → opens the full bot-editor (also prereqs the §5.7 +10 TC "first non-default algorithm" discovery reward)

The key split is branches 3 vs 4 — they reach the same outcome (first-round loss) but need opposite advice. A user who only did the mandatory step-4 training and lost needs *encouragement to train more*; a user who trained hard and still lost needs a *different tool*. Giving "train again" to a heavy-training user who just lost is a retention-killer.

Config location: `backend/src/config/coachingCardRules.js` — plain JS array of rule objects:

```
[
  { match: (ctx) => ctx.finalPosition === 1,
    title: '[1st] Champion!',
    body:  'Your bot is ready for a bigger test...',
    ctaLabel: 'Enter Rookie Cup',
    ctaAction: 'rookieCupClone' },
  { match: (ctx) => ctx.finalPosition === 2, ... },
  // ...
]
```

Predicates are JS, not SystemConfig JSON — rule matching requires code. Editing the rules is a PR, not an admin action. Same pattern as `curriculumNamePools.js`.

Copy above is placeholder. Final strings are a brand-voice decision — rewrite to match the platform's tone before implementation.

Why this is the right shape for v1

- **Four branches = the minimum that maps to distinct next actions.** Fewer (e.g. binary win/lose) would give champions and first-round losers the same advice. More (e.g. sub-branches for "won via sweep") would be distinctions without different CTAs.
- **Rule-based, not AI.** The inputs are small and discrete; ML here is pointless complexity.
- **CTAs wire directly into next actions.** Not generic advice but clickable paths: "enter Rookie Cup" clones one; "switch algorithm" opens the full editor; "train again" opens the training UI. Each CTA is the *start* of a Specialize-phase behavior the user should learn.
- **Curriculum-only scope.** Coaching is a teaching affordance; Specialize users have graduated and get the normal recommendation stack. Layering coaching on Specialize tournaments would create dual noise channels.

5.6 Journey schema migration

- Wipe `journeyProgress` for all users (safe pre-launch)
- Remove the three client-triggerable steps from `POST /guide/journey/step` — all new steps are server-detected
- Add migration to attach `guide:curriculum_complete` and `guide:specialize_start` events to the journey service's emitter

5.7 Discovery rewards — archetype-activating micro-rewards

Beyond the two journey-completion rewards (§4), the Guide grants **one-shot TC rewards** the first time a user engages with a meaningful Specialize-phase feature. These extend the "earn something" feel past the journey so post-graduation doesn't feel empty, and they validate each archetype as the user begins expressing preference.

Event	Reward	Archetype activated	Trigger
First Specialize recommendation acted on	**+10 TC**	— (universal)	User clicks a Specialize card's CTA for the first time (any bucket)
First win in a non-Curriculum tournament	**+25 TC**	Competitor	Tournament completes with `finalPosition == 1` AND `isCup == false` AND user's bot is the winner
First bot trained with a non-default algorithm	**+10 TC**	Trainer	Training run completes on a bot whose algorithm is not the Quick Bot default (§5.3)
First template cloned	**+10 TC**	Designer	User completes a tournament-template clone action the first time

Characteristics of discovery rewards

- **One-shot per user, per event.** Cloning a second template does not grant +10 again. The reward is for *discovery*, not repetition.
- **Admin-configurable.** Each reward has a SystemConfig key (`guide.rewards.discovery.firstSpecializeAction`, `guide.rewards.discovery.firstRealTournamentWin`, `guide.rewards.discovery.firstNonDefaultAlgorithm`, `guide.rewards.discovery.firstTemplateClone`) — see §8.4.
- **Tracked per-user.** A `discoveryRewardsGranted` string-array field on the user preferences JSON stores which events have already been rewarded. Prevents double-payment and makes the set auditable.
- **Explorer intentionally excluded.** Explorer is meant to emerge from natural consumer behavior (spectate, follow, browse). Paying a user +5 TC to follow a bot rewards the *action*, not the *interest* — which defeats the point of letting Explorer emerge organically. If post-launch data shows Explorer is a distinct meaningful cohort and we want to nudge it, we can add an Explorer-flavored discovery reward in v2.

UI

Discovery rewards use the same `JourneyCompletePopup` celebration mechanism as the journey rewards — one-time modal at grant time with the "+N TC" copy and a short explanation of what the reward is for ("First Specialize action — the Guide is working for you!").

Rationale for the specific four

- #1 (universal) — catches *every* user at the moment they first engage with Specialize. Without this, post-journey can feel like "okay, now what?" — this makes the first interaction feel rewarded.
- #2 (Competitor, +25 — biggest) — the real "my bot is good" moment. Cup doesn't count because it's designed to be beatable. The first genuine field win is a proud moment worth celebrating.
- #3 (Trainer, +10) — rewards exploring depth. A user who only ever uses the default algorithm never touches the platform's ML range; this nudges them past that.
- #4 (Designer, +10) — rewards exploring breadth. Template cloning is the fastest way to see what's possible bot-design-wise.

Three archetype-activating rewards + one universal welcome. Keeps the discovery loop alive without reward inflation.

5.8 Rookie Cup — second-tournament template + clone (Specialize Competitor rec #1)

The Rookie Cup is the user's *second* tournament — surfaced as the top Competitor-bucket recommendation (§6.4) after Curriculum graduation. Purpose: take a user who just completed the 4-entrant Curriculum Cup and put them in a meaningfully harder field with a genuine boss to aim at.

Reuses the template + clone pattern from §5.4. Differences from Curriculum Cup:

	Curriculum Cup	Rookie Cup
Template name	"Curriculum Cup"	"Rookie Cup"
Slot count	4	8
Composition	2 Rusty + 1 Copper	4 Rusty + 2 Copper + 1 Sterling
Sterling	none	1, seeded to opposite bracket arm
Match count	3	7
Typical runtime	30–60 sec	~2 min
Spawn trigger	Curriculum step 6 CTA	Specialize Competitor recommendation #1 CTA
`isCup` flag	`true`	`true`
ELO impact	none	none
Retention	30 days	30 days

Endpoint: `POST /api/v1/tournaments/rookie-cup/clone` — thin wrapper targeting the Rookie Cup template.

Name pools shared with Curriculum Cup (§5.4). The Sterling pool is exercised here (unused by Curriculum). Draw-without-duplicates-within-a-tournament rule applies.

Flag rename: the originally-proposed `isCurriculum` becomes `isCup`. Introducing a second cup tier means the original name is semantically wrong for Rookie (which is *post*-Curriculum). `isCup` keeps a single boolean covering all curated-field

tournaments regardless of tier. Tier-specific queries use the template reference (`templateName == "Rookie Cup"`) — no new column needed. Pre-launch, the rename is harmless.

5.9 Deterministic bracket seeding (new tournament infrastructure)

Rookie Cup's "Sterling in the opposite bracket arm" requires deterministic seed placement. The existing tournament system does not support this — it shuffles participants before bracket generation. We're adding it as a general capability available to any template, not a Guide-specific feature.

New field: `seedingMode` enum on `Tournament` + template

- `'random'` (default, current behavior) — participants shuffled before bracket is generated
- `'deterministic'` (new) — participants placed at their pre-assigned `slotIndex`, bracket is generated respecting that order

When a template sets `seedingMode = 'deterministic'`, the `slotIndex` assigned to each template participant is carried through the clone to the generated bracket. For Rookie Cup specifically:

slotIndex	Role	Bracket arm
1	User's bot	A (top half)
2	Rusty	A
3	Rusty	A
4	Copper	A
5	Rusty	B
6	Rusty	B
7	Copper	B
8	**Sterling**	B — furthest from user

With SINGLE_ELIM and this seeding, the user climbs through Arm A (two Rusty matches, then Copper); Sterling climbs through Arm B (the same progression); they can only meet in the final. Quarter-finals pit user vs. slot 8 descendent of their own arm, not the Sterling one.

Scope of the change

- `Tournament` and template tables: add `seedingMode` enum column, default `'random'` (existing rows unchanged)
- `TournamentParticipant`: `slotIndex` field already exists — we just respect it when `seedingMode == 'deterministic'` instead of ignoring it
- Bracket generator: a new code path that places participants by `slotIndex` instead of shuffling
- Admin UI: single toggle on the template-editor to set `seedingMode`
- E2E spec: verify deterministic placement survives clone + actually places Sterling in the target slot
- Unit tests: new branch coverage on the bracket-generation function

Why this is a general feature, not a one-off Guide hack

- Future tournaments may want hand-seeded brackets (e.g. a "Championship" with admin-curated match-ups)
- Determinism is also useful for test fixtures — an E2E test can pin a specific bracket
- Once built, additional Cup tiers (Veteran Cup, etc.) can reuse it for free

6. Specialize Phase — Four Buckets

After Curriculum, the Guide switches from linear steps to **faceted recommendations** based on activity-derived archetypes.

6.1 Archetype detection

Each user gets four scores, normalized to [0, 1]. Scores are recomputed on activity change (bot created, training completed, tournament completed, match spectated, bot followed).

Bucket	Score formula (conceptual)	Why the constant
Designer	`bots_created / max(bots_created, D)` weighted by `1 / (avg_training_runs_per_bot + 1)`	`D = 3`: the point at which "creating bots" has clearly become a repeated behavior, not a one-off experiment
Trainer	`total_training_runs / max(total_training_runs, T)` weighted by `1 / (bots_created + 1)`	`T = 10`: roughly 30 min – 2 hrs of engagement depending on algorithm; signals sustained investment rather than the one mandatory step-4 training run
Competitor	`tournaments_entered / max(tournaments_entered, C)` weighted by `1 + wins_bonus`	`C = 3`: past Curriculum Cup + Rookie Cup + one self-chosen tournament — the user has explicitly chosen to compete at least once beyond the guided path
Explorer	`(spectated_matches + follows_count + rankings_views) / (producer_signals + consumer_signals + 1)` with a minimum-activity floor	Minimum floor `F = 5` consumer events — prevents a brand-new user with zero activity from auto-classifying as Explorer via a divide-by-small-number technicality

Dominant bucket = highest score. If top two are within `E = 0.1` of each other, the user is considered "balanced" and gets recommendations from both.

All five constants (D, T, C, F, E) are admin-tunable via SystemConfig — see §8.4. Post-launch tuning is expected within the first 90 days as real user distributions emerge; the admin-configurable setup turns tuning from a code-deploy cycle into a config change.

The §2 measurement dashboard surfaces individual archetype score histograms per bucket (not just dominant-assignment counts), so we can detect when a constant is miscalibrated. Example: if >95% of users have Designer scores between 0.0 and 0.1 with only 1% above 0.5, `D = 3` is too high — the Designer bucket is effectively dead. Histograms give us the signal; admin-tuning gives us the lever. Without instrumenting the distribution, we'd be tuning blind.

Starting state at graduation. Every user who completes Curriculum has exactly one meaningful action in each of the three producer buckets (1 bot created, 1 training run, 1 tournament completed) and zero consumer activity. Scores are approximately (0.3, 0.3, 0.3, 0.0) — **a balanced producer**. Explorer starts at zero by design; it is an **emergent archetype**, not a default one, and only begins accruing signal when the user spectates a match they are not participating in, or follows a bot.

In practice, Curriculum behavior biases the start — a user who created 3 bots before moving to training graduates with a slight Designer lean; a user who trained their one bot 5 times before sparring graduates with a slight Trainer lean. The minimum-viable path (one action per step) produces the pure balanced default above.

There is **no Competitor cold-start bias**. Showing three Competitor cards to someone who just completed their first tournament is redundant ("you just did that"). A balanced producer surface lets the user self-select their next direction, and their pick becomes the first real archetype signal. The system is designed so the bucket recommender works from day one rather than guessing.

6.2 Designer bucket

Archetype: builds many bots, iterates on variety rather than depth.

#	Recommendation	Prereq	Disqualifier
1	Try a different persona	≥ 2 bots with same persona	—
2	Clone a template	At least one public template exists	User has < 2 bots
3	Try a new algorithm family	Hasn't used ≥ 3 distinct algorithm kinds	—
4	Build a fully custom bot via SDK	Has a bot with ≥ 2 training runs	User in cold-start

Global disqualifier for Designer: if `tournaments_entered == 0`, suppress all Designer recommendations and substitute a Competitor recommendation. Rationale: more bot variety is pointless if none of the bots ever compete.

6.3 Trainer bucket

Archetype: picks a bot and trains it repeatedly; cares about performance over variety.

#	Recommendation	Prereq	Disqualifier
1	Try a different training algorithm	Has only used 1 algorithm family	—
2	Compare training runs side-by-side	≥ 2 runs on same bot	—
3	Tune hyperparameters	Has completed a default run	Fewer than 2 training runs total
4	Read the Game SDK Developer Guide	—	—
5	Long/overnight training run	Has completed ≥ 3 short runs	—

Global disqualifier for Trainer: if the trained bot has never played a tournament match, suppress Trainer recommendations and substitute a Competitor recommendation. Rationale: training without competition gives no feedback loop.

6.4 Competitor bucket

Archetype: enters tournaments, watches leaderboards, cares about wins.

#	Recommendation	Prereq	Disqualifier
1	Enter the **Rookie Cup** (8-entrant, includes Sterling)	Curriculum Cup completed; Rookie Cup not yet attempted	—
2	Enter a larger-field public tournament	Rookie Cup completed; has only entered Cup-tier fields	—
3	Follow a top-ranked bot	—	—
4	Create your own tournament	Entered ≥ 3 tournaments	—
5	Try a different bracket type	Only tried SINGLE_ELIM	User has ≤ 1 tournament entered
6	Set up a recurring tournament	Has created ≥ 1 tournament	—

Recommendation #1 (Rookie Cup) is the headline Specialize-phase experience for a fresh Curriculum graduate. Clicking the card clones a fresh Rookie Cup (8-entrant, Sterling boss) — see §5.8. It is the single highest-signal recommendation in the Competitor bucket and the primary path from "I completed the tutorial" to "I beat a real field."

Global disqualifier for Competitor: if the user has lost their last 3 tournament matches in a row, suppress Competitor recommendations and substitute a Trainer recommendation ("Your bot could use more training"). Rationale: recommending "enter a bigger tournament" after repeated losses is demoralizing and counterproductive.

6.5 Explorer bucket

Archetype: consumer-first — spectates matches, follows bots, browses rankings. Primary engagement is watching the platform happen, not producing. A legitimate terminal state, not a failure mode.

#	Recommendation	Prereq	Disqualifier
1	Follow a top-ranked bot	Not following ≥ 3 bots	—
2	Watch a live tournament	Any IN_PROGRESS tournament exists	—

#	Recommendation	Prereq	Disqualifier
3	Replay a champion bot's past matches	Has followed ≥ 1 bot	—
4	Subscribe to a recurring tournament's schedule	Recurring tournaments exist	—
5	Curate a watchlist of favorite bots	Has followed ≥ 3 bots	—

Global disqualifier for Explorer: once per session, substitute one Competitor recommendation ("Your turn — try entering with any bot"). Rationale: Explorer is a legitimate archetype we want to support, but the North Star metric cares about tournament participation; one gentle seed-planting card per session keeps the producer funnel alive without being pushy.

6.6 Why four buckets (and not three or five)

- **Designer / Trainer / Competitor** cover the producer side of the platform (build → train → compete). Each maps to a distinct phase of the bot lifecycle.
- **Explorer** covers the consumer side (watch / follow / browse) — drivers of platform vibrancy, audience for top bots, social gravity. Folding Explorer into Competitor (as "Follow a top-ranked bot") undersells the archetype; some users will never build but will spectate deeply, and that's a legitimate cohort.
- **A 5th bucket was not added.** Candidates (e.g. "Social" for tournament-creators, "Theorist" for SDK/algorithm readers) don't have enough distinct signal from the existing four. Tournament-creators look like advanced Competitors; SDK readers look like advanced Trainers or Designers.

The four-bucket design separates the **producer lifecycle** (three buckets) from the **consumer archetype** (one bucket):

- **Producer lifecycle**: Designer → Trainer → Competitor. Three phases of producing value — design the bot, train it, compete with it. Users typically flow left-to-right but can specialize in one phase indefinitely.
- **Consumer archetype**: Explorer. Parallel to the lifecycle — a user can be Explorer-dominant from signup and never cross into production, or oscillate between Explorer and Competitor (watch, then try, then watch again).

A user's score in each bucket is independent; a deeply engaged user might be high in Competitor *and* Explorer simultaneously (the person who enters tournaments *and* watches top-bot replays to learn).

7. Scoring & Surfacing

7.1 Card surfacing rules

- Guide renders **up to 3 cards** at a time
- Default mix: **2 from dominant bucket + 1 from secondary bucket** (for variety — prevents the Guide from becoming monotonous)
- Balanced users (top-two scores within 0.1): **1 from each top bucket + 1 wildcard from a third bucket**
- **Fresh graduates** (just completed Curriculum, no producer bucket dominant yet): **1 Designer + 1 Trainer + 1 Competitor** — one card from each producer bucket, letting the user self-select their next direction. Explorer is not surfaced at graduation (no Explorer signal exists yet); it appears naturally once the user begins spectating or following.
- **Explorer-dominant users** always include **at least 1 Competitor card** in the surface — enforced by the Explorer global disqualifier (§6.5). This seeds the producer funnel for long-term Explorers without hijacking their preferred mode.

7.2 Ordering within a bucket

Recommendations are ordered by:

1. **Prereqs met** (filter)
2. **Not already completed** (filter)
3. **Disqualifiers not active** (filter)
4. **List order** in §6.2–6.4 (the catalog is itself authored in "next-best" order)

The first eligible recommendation is surfaced. No ML scoring in v1 — the curated list order is the heuristic.

7.3 Dismissal and re-engagement

Normal behavior

- **Dismiss a card**: suppressed for 7 days, then re-eligible with a small ranking penalty (-0.2 on its score). We do not permanently remove recommendations; users change their minds.
- **Complete a recommended action**: permanently completed, removed from the catalog for that user.
- **No action in 14 days**: Guide emits a `guide:notification` event via socket. The GuideOrb pulses to pull the user's eye. This is the *only* automatic push — no modals, no emails.

7.4 Stagnation handling — decay and exhaustion

The Guide detects two distinct "user isn't engaging" failure modes and responds differently to each. Without this, a user who dismisses every card forever sees three stale recommendations every session, and the Guide becomes a broken-looking appliance.

Mode A: Dismissal-only streak (rejection without action)

Trigger: user has performed only `dismiss` actions (no `ctaAction` clicks, no recommendation completions) for `guide.stagnation.dismissalStreakDays` (default **30 days**).

Response — the Guide **decays to quiet**:

- Specialize card stack drops from 3 cards to `guide.stagnation.decayCardCount` (default **1 card**)
- GuideOrb stops pulsing on inactivity (§7.4's 30-second-to-pulse escalation is suspended)
- Card content unchanged — same rules, same scoring — just less visual weight
- Reverts instantly the moment the user does *anything* non-dismissive: clicks a card CTA, completes a recommendation, acknowledges a notification. One signal is enough — we were wrong about them being disengaged.

This is lighter than hiding the Guide entirely. The user can still open the panel and see a card if they want to; the Guide just stops fighting for their attention.

Mode B: Catalog exhaustion (completion, not rejection)

Trigger: no eligible recommendations remain in the user's dominant bucket — every available one is either already completed or actively disqualified.

Response — **wildcard mode**:

- A single "You've explored [bucket] deeply. Try something different?" card appears
- Content is a **wildcard recommendation** pulled at random from any non-dominant bucket (ignores normal surfacing rules)
- Rotated every `guide.exhaustion.wildcardRotationDays` (default **7 days**)
- Exits wildcard mode automatically if the user enters a new bucket (activity causes archetype shift) or completes the wildcard recommendation

Wildcard mode is the "power user" end-state — users who have *actually done everything* shouldn't be told "keep trying" with stale content.

Why distinguish these two modes

A user who dismisses every Designer card for 30 days might just hate Designer recommendations (mode A — decay). A user who has *completed* every Designer recommendation has a different problem (mode B — exhaustion, surface new territory). Treating them the same wastes both signals.

Admin-configurable via:

- `guide.stagnation.dismissalStreakDays` = `30`
- `guide.stagnation.decayCardCount` = `1`
- `guide.exhaustion.wildcardRotationDays` = `7`

7.5 Progressive intensity

When a transition happens (e.g. Hook → Curriculum, or Curriculum → Specialize):

1. **JourneyCard updates inline** — the user is already looking at it
2. **If no interaction in 30 seconds**, GuideOrb pulses (gentle animation, no sound)

3. **If still no interaction after 2 minutes**, slide-in panel from bottom of screen (non-modal)
4. **Hard popup reserved for rewards** (+20 TC, +50 TC) — these are celebrations, not nudges

8. Backend Architecture

8.1 New service files

- `backend/src/services/recommendationService.js` — Specialize-phase scoring and card generation. One public function: `getRecommendations(userId) → Card[]`.
- `backend/src/services/userActivitySummary.js` — signals layer. One public function: `summarize(userId) → { botsCreated, trainingRuns, tournamentsEntered, ... }`. Cached per-session.
- `backend/src/config/featureCatalog.js` — the authored catalog for all three buckets. Plain JS data, no DB.

8.2 Updated service files

- `backend/src/services/journeyService.js` — rewritten with the 7-step spec from §4. Trigger functions hook into existing events (game completed, bot created, training completed, tournament completed).
- `backend/src/routes/guide.js` — remove `POST /journey/step` (no more client-triggered steps). Add `GET /api/v1/guide/recommendations` which wraps `recommendationService.getRecommendations`.

8.3 New endpoints

Verb	Path	Auth	Purpose
GET	`/api/v1/guide/recommendations`	required	Returns current recommendation cards for the user
POST	`/api/v1/guide/recommendations/:id/dismiss`	required	Dismiss a card (7-day suppress)
POST	`/api/v1/tables/demo`	required	Demo Table macro (§5.1)
POST	`/api/v1/bot-games/practice`	required	Public Spar (§5.2)
POST	`/api/v1/bots/quick`	required	Quick Bot wizard endpoint (§5.3)
POST	`/api/v1/tournaments/curriculum-cup/clone`	required	Curriculum Cup template clone (§5.4)
POST	`/api/v1/tournaments/rookie-cup/clone`	required	Rookie Cup template clone (§5.8)
POST	`/api/v1/guide/guest-credit`	required	Credits Hook steps 1–2 on signup from guest localStorage events (§3.5.3)

8.4 Schema additions

Three new flags/fields across existing tables to support the Guide features. All default to `false` / `null` so existing data is unaffected.

Table	Column	Type	Purpose
`Table`	`isDemo`	`boolean default false`	Marks a Hook-phase demo table for privacy scoping + GC (§5.1)
`BotGame`	`isSpar`	`boolean default false`	Marks a user-initiated practice match for 30-day retention (§5.2)

Table	Column	Type	Purpose
`Tournament`	`isCup`	`boolean default false`	Marks any curated-field Cup tournament (Curriculum Cup §5.4 or Rookie Cup §5.8) for privacy, 30-day GC, ELO exclusion. Tier is derived from the cloned template reference — no separate column needed.
`Tournament` + template	`seedingMode`	`enum('random', 'deterministic') default 'random'`	General deterministic-bracket-seeding support (§5.9). Existing behavior unchanged; Rookie Cup template sets this to `'deterministic'`.
`User`	`isTestUser`	`boolean default false`	Excludes internal/admin/QA/dev accounts from all metrics aggregations. Set `true` on creation for admins, seed-script accounts, and internal email domains. See §2 pollution-prevention.
`MetricsSnapshot`	`date`, `metric`, `value`, `dimensions (JSON)`	new table	Daily aggregate snapshots for the dashboard (§2 Measurement)

Each flag is mutually exclusive with the others (a row is at most one of demo / spar / curriculum). A Prisma migration will add them in one step.

Sweep job updates — existing `tournamentSweep.js` gets a new phase scanning `isCup = true AND completedAt < NOW() - 30 days` for soft-delete cascade. The table/game sweep picks up demo + spar cleanup per §5.1–§5.2.

Admin-configurable settings (SystemConfig keys)

Guide tunables live in the existing `SystemConfig` table (accessed via `_getSystemConfig` pattern in `creditService.js`). Admins can adjust these through the admin-config UI without a code deploy:

Config key	Default	Description
`guide.rewards.hookComplete`	`20`	TC awarded at Hook step 2 completion
`guide.rewards.curriculumComplete`	`50`	TC awarded at Curriculum step 7 completion (replaces the current hardcoded `JOURNEY_COMPLETE_TC`)
`guide.rewards.discovery.firstSpecializeAction`	`10`	TC awarded the first time a user acts on any Specialize recommendation (§5.7)
`guide.rewards.discovery.firstRealTournamentWin`	`25`	TC awarded on first non-Curriculum tournament win (§5.7)
`guide.rewards.discovery.firstNonDefaultAlgorithm`	`10`	TC awarded on first training run using a non-default algorithm (§5.7)
`guide.rewards.discovery.firstTemplateClone`	`10`	TC awarded on first tournament-template clone action (§5.7)
`guide.cup.sizeEntrants`	`4`	Number of slots in the Curriculum Cup (user + N-1 system bots)
`guide.cup.retentionDays`	`30`	Days before Curriculum Cup GC
`guide.demo.ttlMinutes`	`60`	Hard TTL for demo tables

Config key	Default	Description
`guide.quickBot.defaultTier`	`"novice"`	Minimax tier for a freshly-created Quick Bot (Rusty-equivalent — random moves). §5.3
`guide.quickBot.firstTrainingTier`	`"intermediate"`	Minimax tier after step-4 first-training-run completes (Copper-equivalent — blocks/wins). §5.3
`guide.archetypes.designer.normalizationThreshold`	3	Designer formula's `D` constant. Raise if the bucket is too easy to enter; lower if it's empty. §6.1
`guide.archetypes.trainer.normalizationThreshold`	10	Trainer formula's `T` constant. Training runs needed to approach full Trainer saturation. §6.1
`guide.archetypes.competitor.normalizationThreshold`	3	Competitor formula's `C` constant. Tournaments-entered for full Competitor saturation. §6.1
`guide.archetypes.explorer.minActivityFloor`	5	Minimum consumer events before Explorer can score non-zero. Prevents divide-by-small-number auto-classification. §6.1
`guide.archetypes.balancedTopTwoEpsilon`	0.1	Threshold for "balanced user" classification (top two bucket scores within ε). §6.1
`guide.stagnation.dismissalStreakDays`	30	Days of dismissal-only behavior before the Specialize surface decays to quiet mode (§7.4)
`guide.stagnation.decayCardCount`	1	Number of cards to show during stagnation decay (normally 3). §7.4
`guide.exhaustion.wildcardRotationDays`	7	Days before the exhaustion-mode wildcard recommendation rotates. §7.4
`metrics.internalEmailDomains`	`[]` (empty JSON array)	Email domains that cause new accounts to be flagged `isTestUser = true` on creation. Example: `["@xo-arena.internal", "@callidity.com"]`. §2
`guide.metricsSnapshot.retentionDays`	`null`	Retention window for `metricsSnapshot` rows. `null` = indefinite (default). Set to an integer (e.g. `730`) to enable trimming. Aggregates are tiny (~150 KB/day), so indefinite is the right default; this key exists as a future safety valve.
`guide.spar.retentionDays`	30	Days before spar match GC
`guide.recommendations.dismissSuppressDays`	7	Suppression window after a Specialize card is dismissed
`guide.recommendations.dismissScorePenalty`	0.2	Score penalty applied when a dismissed card re-appears
`guide.inactivityNudgeDays`	14	Days of no Specialize-action before the orb pulses

Tunables are read on each use (not cached aggressively) so admin changes take effect within seconds. Each key has a unit test asserting the default value matches the documented default above — prevents silent drift between doc and code.

8.5 Server-authoritative, client-rendering

- All journey state lives in `journeyProgress`
- All recommendations are generated server-side per request
- Client does no scoring, no trigger inference. It just renders what the server returns.

This is a deliberate change from the current hybrid model, which had the client trigger steps 2, 4, and 7. That hybrid made testing painful and caused state divergence. Removing it is a direct cost — the server must now detect e.g. "user visited /faq" — but for v1 we're dropping the "visit the FAQ" step entirely (FAQ knowledge is content, not a milestone).

9. UI Changes

9.1 `JourneyCard` rewrite

Component accepts a `phase` prop: `'hook' | 'curriculum' | 'specialize'` and renders a different visual shape for each — **three tiers of decreasing visual weight as the user progresses**.

Hook phase — Hero card (1 at a time, maximum visual weight)

- Single large card filling the Guide panel (~340px wide, substantial height)
- Prominent imagery or iconography
- Large CTA button
- No preview of future steps — Hook is about "do this one thing, it's easy"
- Not dismissible (it's the critical path)

Curriculum phase — Hero card + step checklist

- The current step renders as a hero card (same size as Hook)
- Above or below the hero, a compact 5-row checklist of all Curriculum steps:
  - Completed steps shown with a checkmark, dimmed text
  - Current step highlighted (matches the hero)
  - Future steps shown as plain dimmed rows with their title visible (no CTA, no imagery)
- Rationale: the user has committed to a multi-step journey (create → train → spar → compete → result). Unlike Hook, they benefit from seeing the ladder they're climbing. Checklist gives progress + preview without fragmenting focus (the hero is still the one "do this now" surface).

Specialize phase — Recommendation card stack (medium weight)

- Up to 3 cards stacked vertically
- Each card: title, short body with reasoning, CTA, small dismiss "×" button
- Smaller than hero cards (suggestions, not commands) but larger than tiles
- Dismiss fires 7-day suppression per §7.3

9.2 `GuideOrb` retained

- No visual change to the idle state
- New pulse animation for the 30-second inactivity nudge
- New "3 cards waiting" badge when Specialize has 3 unread recommendations

9.3 `SlotGrid` — locked during journey, unlocked at graduation

The 8-slot user-configured shortcut tray is **a graduation reward**, not a default surface.

During Hook and Curriculum

- `SlotGrid` is **hidden entirely** — not visible, not present as a disabled tab, not discoverable
- `GuidePanel` renders only the current phase's view (hero card for Hook, hero + checklist for Curriculum)
- **Why fully hidden, not read-only**: a disabled "Shortcuts" tab teaches the user "I can't do this yet," adding noise without value. Hiding is simpler and makes graduation more rewarding.

At Curriculum graduation (step 7 completion) — multiple things unlock in the same moment:

1. `JourneyCompletePopup` fires with the +50 TC celebration
2. `GuidePanel` sprouts tabs: **"What's Next"** (Specialize cards, default tab) and **"Shortcuts"** (SlotGrid tab)

3. `SlotGrid` auto-populates with the curated `postJourneySlots` defaults — 4–6 pre-pinned tiles, remaining slots empty for user additions
4. Tile tray becomes fully editable: drag to rearrange, remove tiles, add from the `slotActions` catalog

Edge cases

- **Skip onboarding** (if a "Skip" affordance is ever added): `journeyProgress` frozen at current step, SlotGrid + Specialize both unlock. Opting out is not being gated forever.
- **Restart onboarding** (existing `POST /guide/journey/restart`): `journeyProgress` resets, user re-enters Hook — **but SlotGrid stays unlocked and keeps the user's current tile configuration.** Once earned, always available. Otherwise a refresh-tour would destroy customization, which is user-hostile.

Why tier this way

The visual hierarchy (Hero → Recommendation Card → Tile) maps to the user's familiarity with the platform. Large cards demand attention when the user doesn't yet know what to do; small tiles are efficient once the user knows their daily drivers. The Specialize phase sits between — recommendations are suggestions, not gates, so they get medium weight.

9.4 Reward celebrations

The existing `JourneyCompletePopup` gets two triggers instead of one (end-of-Hook and end-of-Curriculum). Each uses phase-appropriate copy and the corresponding TC reward. The popup remains the only modal-blocking UI in the Guide.

9.5 Admin experience

Admins (`ADMIN`, `TOURNAMENT_ADMIN`, `BOT_ADMIN`, support roles) have a different relationship with the platform than regular users — many are operational-only; others want to dogfood the player flow. The Guide handles both cases without forcing the journey on admins who don't want it, and without hiding the Guide from admins who do.

Player Guide visibility — opt-in for admins

- A user-level setting **"Show player Guide (journey + recommendations)"** lives in the Settings page
- **Default:** off for anyone with an admin role; on for everyone else
- When off, the player Guide surface (hero cards, Specialize recommendations, journey progress UI) is hidden — the admin sees only admin-specific surfaces
- When on, the admin sees the normal Hook → Curriculum → Specialize flow alongside their admin tiles
- **Journey progress server-records regardless of toggle state** — if the admin plays a PvAI game, step 1 is marked complete even if they haven't flipped the toggle on yet. Avoids a "surprise, you're halfway through onboarding" moment.

Admin-specific SlotGrid tiles

The `slotActions.js` catalog gains a new `requiredRole` metadata field on each tile:

```
{
  id: 'admin-stuck-tournaments',
  label: 'Stuck tournaments',
  icon: '⚠',
  action: '/admin/tournaments?status=stuck',
  requiredRole: ['ADMIN', 'TOURNAMENT_ADMIN'],  // ← new
}
```

Filtering happens in two places:

1. **Tile-picker UI**: entries with `requiredRole` are filtered out unless the caller's roles satisfy the requirement. **Non-admins never see admin tiles in the "Add tile" picker.**
2. **SlotGrid render**: if a pinned tile's `requiredRole` is no longer satisfied (user lost admin role, layout copied from another user, etc.), the tile renders as hidden — does not surface to the viewer.

Backend endpoints already enforce admin roles via existing middleware (`requireTournamentAdmin` etc.), so the frontend filter is UX — even if a mis-pinned admin tile somehow got clicked, the admin route would 403. Security is defense-in-depth.

Initial admin tile set for v1 (lives in `slotActions.js`):

Tile	Route	Required role
Stuck tournaments	`/admin/tournaments?status=stuck`	`TOURNAMENT_ADMIN`
Runaway guard dashboard	`/admin/tournaments#runaway`	`TOURNAMENT_ADMIN`
Incident log	`/admin/incidents`	`ADMIN`
User support queue	`/admin/support`	`ADMIN`
System config editor	`/admin/config`	`ADMIN`
Guide metrics dashboard	`/admin/guide-metrics`	`ADMIN`

Set can grow via PR as admin pain points emerge.

Admin-supplemented default tiles at graduation

Admins who graduate Curriculum (or have `postJourneySlots` applied) get their standard 4–6 default tiles **supplemented with 2 role-appropriate admin tiles** (the two most-useful for their specific admin role). Keeps the Guide immediately useful for admins who dogfood — no manual admin-tile discovery required. If they don't want admin tiles in their grid, they remove them like any other tile.

What's deliberately not in v1

A separate **admin-specific recommendation stack** (parallel to Specialize recommendations but for operational items like "3 tournaments stuck in IN_PROGRESS > 24h") was considered and deferred. Admin tiles give fast access to admin pages — the "proactive admin alerts" concept is a different feature that overlaps with what a proper admin dashboard already provides. Revisit post-launch if admins report recurring "I keep forgetting to check X" pain; until then, tiles are enough.

10. Testing

The Guide is load-bearing for the engagement funnel — a silent regression here directly damages the North Star metric. Three test layers, matching the project's existing structure (`backend/**/__tests__/`, `landing/**/__tests__/`, `e2e/tests/`). Per repo convention (see `feedback_tests_before_completion.md`), tests are written alongside each feature and block merge.

10.1 Backend regression / unit tests

Location: `backend/src/**/__tests__/`.

Existing files to extend

- `journeyService.test.js`
  - Each of the 7 steps: trigger fires → step completed (idempotent)
  - Step ordering: step N cannot complete before step N-1 (or explicitly document which steps skip)
  - Rewards: +20 TC granted exactly once at step 2 completion; +50 TC exactly once at step 7 completion
  - Graduation: Curriculum → Specialize transition emits `guide:specialize_start` exactly once
  - Restart: `journeyProgress` resets and user re-enters Hook

New files

- `recommendationService.test.js`
  - Fresh-graduate starting state: 1 Designer + 1 Trainer + 1 Competitor, no Explorer card
  - Dominant-bucket mix: 2 dominant + 1 secondary
  - Balanced top-two: 1 + 1 + 1 wildcard
  - Explorer-dominant: Competitor card always present (§6.5 global disqualifier)
  - Every cross-bucket disqualifier has its own test:
    - Designer with 0 tournaments → Competitor substitution
    - Trainer whose bot has never played a tournament match → Competitor substitution
    - Competitor with 3 consecutive losses → Trainer substitution
  - Dismissal: card suppressed for 7 days → re-eligible with −0.2 score penalty
  - Prereq filtering: recommendations with unmet prereqs not surfaced
  - Completed filtering: recommendations the user has already acted on are permanently removed
- `userActivitySummary.test.js`
  - Aggregate counts match raw DB counts (`bots_created`, `training_runs`, `tournaments_entered`, `spectated_matches`, `follows_count`)

  - Cache invalidation on activity event
  - Explorer signals accrue correctly from spectate / follow events
- `featureCatalog.test.js`
  - Every catalog entry has `id`, `bucket`, `title`, `prereqCheck`, `cta`
  - No duplicate IDs across buckets
  - Each bucket has ≥ 3 entries (required for card-surfacing variety rules)

New endpoint tests (one `__tests__/*.test.js` per new route)

- `tablesDemo.test.js` — 201 on valid, 401 unauth, 429 rate limit (5/day), response contains `tableId`
- `sparRoute.test.js` — 201 on valid, 401 unauth, 403 if user doesn't own `myBotId`, 429 rate limit (20/day)
- `botsQuick.test.js` — 201 on valid, 400 on missing persona, all persona choices accepted
- `guideRecommendations.test.js` — 200 returns well-shaped `Card[]`, 401 unauth
- `guideDismiss.test.js` — 204 on valid card id, 404 on unknown, 403 when dismissing another user's card

Metrics job test (for Phase 4)

- `metricsSnapshot.test.js` — aggregate correctness against hand-rolled fixture oracle; idempotent re-run (same UTC date overwrites, never duplicates); backfill script produces identical snapshots for historical days

10.2 UI / component tests (landing)

Location: `landing/src/**/__tests__/` using Vitest + React Testing Library.

- `JourneyCard.test.jsx`
  - Hook phase: renders current step title + CTA
  - Curriculum phase: renders "Step N of 5" progress indicator + CTA
  - Specialize phase: renders up to 3 stacked cards, each with dismiss button
  - CTA click dispatches expected action / navigation
  - Dismiss click calls `dismissCard` with the correct id
- `GuideOrb.test.jsx`
  - Idle: no pulse class
  - Pulse: pulse class applied when the inactivity trigger fires
  - "3 cards waiting" badge visible when `recommendations.length === 3`
- `GuidePanel.test.jsx`
  - Default view is the user's current phase (Hook / Curriculum / Specialize)
  - Shortcuts tab accessible and renders the 8-slot tray
  - Phase transition re-renders the default view
- `JourneyCompletePopup.test.jsx`
  - Fires at end-of-Hook with +20 TC copy
  - Fires at end-of-Curriculum with +50 TC copy
  - Dismisses cleanly without leaving backdrop
- `QuickBotWizard.test.jsx` (new component)
  - 3-step flow: name → persona → confirm
  - Empty name blocks advance
  - Submit issues `POST /api/v1/bots/quick` with the correct payload
- Admin dashboard component tests (`GuideMetricsPage.test.jsx`)
  - Renders North Star, funnel, bucket distribution given fixture data
  - Handles empty-state (no snapshots yet)
  - Cohort slicer filters correctly

10.3 E2E tests — added to the QA suite

Location: `e2e/tests/`, wired into `e2e/qa.mjs` following the pattern established by `tournament-guards.spec.js`.

- `guide-hook.spec.js` — fresh-signup hook flow
  - Sign up a new user
  - Play a PvAI game → step 1 server-recorded
  - Click demo-table CTA → spectate the match → step 2 recorded
  - Assert +20 TC reward popup + phase transition to Curriculum
- `guide-curriculum.spec.js` — end-to-end bot lifecycle

  - Starting from hook-completed fixture
  - Quick Bot wizard creates bot → step 3
  - Training run completes → step 4
  - Spar via public Spar endpoint → step 5
  - Enter matchmaking-suggested tournament → step 6
  - Tournament completes → step 7 + +50 TC reward
  - Assert Specialize phase starts with 3 cards: 1 Designer + 1 Trainer + 1 Competitor
- `guide-specialize.spec.js` — bucket detection + card surfacing
  - Starting from Curriculum-completed fixture
  - Create 3 more bots → Designer-dominant → 2 Designer + 1 secondary surfaced
  - Run 10 training runs on one bot → bucket shifts to Trainer-dominant
  - Dismiss a card → replaced by the next eligible
  - Time-warp +7 days (Playwright route-mock) → dismissed card re-appears
- `guide-explorer.spec.js` — consumer archetype emergence
  - Starting from fresh Specialize phase
  - Spectate 5 matches outside user's own bot
  - Follow 3 bots
  - Assert Explorer-dominant surface + required Competitor card per global disqualifier
- `guide-metrics.spec.js` — admin dashboard smoke
  - Login as admin
  - Visit `/admin/guide-metrics`
  - Assert North Star, funnel, bucket distribution all render
  - Assert at least one trend data point exists after the daily cron has run (fixture seeds it)

Wire into `e2e/qa.mjs` — new menu items in the Playwright section:

```
I('Guide: Hook flow',                  () => pw('guide-hook')),
I('Guide: Curriculum flow',            () => pw('guide-curriculum'),    'TEST_USER_EMAIL'),
I('Guide: Specialize + bucket detect', () => pw('guide-specialize'),    'TEST_USER_EMAIL'),
I('Guide: Explorer archetype',         () => pw('guide-explorer'),      'TEST_USER_EMAIL'),
I('Guide: Admin metrics dashboard',    () => pw('guide-metrics'),       'TEST_ADMIN_EMAIL'),
```

10.4 Fixtures & helpers

- **Graduated-user fixture**: creates a user with 1 bot, 1 training run, 1 completed tournament entry with a `finalPosition`, and `journeyProgress` set to graduated. Lets `guide-specialize.spec.js` and `guide-explorer.spec.js` skip the slow Curriculum setup.
- **Time-freeze helper** for 7-day dismissal tests — Vitest fake timers in unit tests, Playwright route-mock in E2E.
- **Seed bots with contrasting styles** for Demo Table tests (one aggressive, one defensive) to keep the demo visually distinguishable.

10.5 What we actually care about (coverage priorities)

Line-coverage percentages are usually vanity. The real bar:

- **Every global disqualifier in §6 has a dedicated unit test.** These are the rules most likely to introduce dark patterns if they break.
- **Every journey step's trigger has a dedicated unit test.** Step-state-machine bugs compound — fixing one creates another.
- **At least one E2E covers the full fresh-signup → Specialize path** (`guide-hook.spec.js` + `guide-curriculum.spec.js` chained or combined). Catches integration bugs unit tests miss.
- **The dashboard never crashes given empty data.** A dashboard with no data is useful; a dashboard that 500s on load is worse than no dashboard.

10.6 QA/Dev CLI (`um`) enhancements

The existing `um` CLI (`backend/src/cli/um.js`) is a critical QA/dev tool for manipulating user state during testing and demoing. Its `um journey` subcommand is step-aware but was designed for the old linear-7-step model; the new phased journey + Specialize state + discovery rewards + `isTestUser` flag require matching CLI support or QA and dev work becomes painful.

Additive, backward-compatible changes — nothing breaks that currently works.

Enhance `um journey` — phase shortcuts + richer display + deeper reset

```
um journey <user> --phase hook           # 0000000 (fresh)
um journey <user> --phase curriculum     # 1100000 (Hook done, ready for step 3)
um journey <user> --phase specialize     # 1111111 (graduated) + grants both rewards
um journey <user> --graduate             # alias for --phase specialize
```

Default output shows phase label:

```
"joe"  [●●●○○○○]  1110000  (3/7)  [Curriculum]  active
```

`--reset` now clears journey state *and* the related side-state: `discoveryRewardsGranted`, stagnation tracking, SlotGrid slots (re-locks), Hook/Curriculum reward grant markers. Effectively "brand-new user."

New `um specialize` — manipulate Specialize-phase state

```
um specialize <user> --bucket designer|trainer|competitor|explorer
              [--dismissal-streak <days>]
              [--wildcard-mode]
              [--reset]
```

- `--bucket <name>` seeds synthetic activity counts so `userActivitySummary` evaluates the user as that bucket-dominant (useful for testing bucket-specific recommendation surfacing without actually creating 3 bots first)
- `--dismissal-streak <days>` writes N dismiss events dated N days ago, triggering the §7.4 decay state
- `--wildcard-mode` marks all recommendations in the user's dominant bucket as completed, triggering the §7.4 exhaustion state
- `--reset` clears Specialize state to "fresh graduate balanced producer"

New `um rewards` — manipulate reward-grant markers independently of TC balance

```
um rewards <user>                                          # show reward state
um rewards <user> --grant hook                             # grant +20 TC + mark as granted
um rewards <user> --grant curriculum                       # grant +50 TC + mark as granted
um rewards <user> --grant discovery:firstRealTournamentWin
um rewards <user> --revoke discovery:firstTemplateClone
um rewards <user> --reset                                  # clear all markers (TC NOT refunded)
```

Revoking resets the `discoveryRewardsGranted` entry *without* touching `creditsTc` — lets QA test "what if this event fires again?" without re-running the whole flow.

New `um testuser` — manipulate the `isTestUser` flag

```
um testuser <user> --on                # flag as test user (excluded from metrics)
um testuser <user> --off               # unflag (counts in metrics)
um testuser --list                     # list all users with isTestUser = true
um testuser --audit                    # list users who should probably be flagged but aren't
                                       # (admin role, internal email domain, seed account, etc.)
```

The `--audit` subcommand catches drift where internal accounts escaped auto-flagging.

Enhance `um status` — add 5 new fields to the existing user-detail output

- Phase (Hook / Curriculum / Specialize)
- Dominant bucket (Specialize only)
- `isTestUser` flag
- Discovery rewards granted (list of event names)
- Current `creditsTc` balance

Test-scenario shortcuts — what QA can now do in one command

Scenario	Command
Fresh Hook user	`um journey alice --phase hook`
Hook complete, Curriculum starting	`um journey alice --phase curriculum`

Scenario	Command
Ready for Rookie Cup (fresh graduate)	`um journey alice --graduate`
Designer-dominant Specialize user	`um journey alice --graduate && um specialize alice --bucket designer`
Stagnant user (decay mode)	`um journey alice --graduate && um specialize alice --dismissal-streak 35`
Exhausted user (wildcard mode)	`um journey alice --graduate && um specialize alice --wildcard-mode`
Excluded from metrics (internal account)	`um testuser alice --on`
Catch drift in test-user flagging	`um testuser --audit`

Commands live alongside existing ones in `backend/src/cli/commands/`. Unit tests per file (the existing pattern) — each new command gets its own test file asserting state transitions are correct.

---

11. Rollout Plan

Four phases, each independently shippable.

Phase 0 — Visitor → Registered User (ships first; everything else depends on signups)

- Landing page redesign: live bot-vs-bot demo hero (reuses §5.1 Demo Table infrastructure)
- Progressive CTA ladder: Watch / Play PvAI / Build your own bot
- Guest mode in `landing/` client: localStorage tracking, Hook step 1 + 2 event recording
- `POST /api/v1/guide/guest-credit` endpoint + signup client integration
- Deferred email verification: post-signup users land directly in Curriculum step 3; soft banner nudges email verification; only tournament entry is gated
- Contextual signup copy: "Build your first bot" variant when modal opens from the "Build a bot" CTA
- Measurement hooks: landing conversion rate, guest progression funnel, time-to-signup, Hook-credited-at-signup rate, signup method split
- Smoke test: a fresh visitor can go landing → play PvAI → watch demo → click "Build a bot" → sign up → land in Quick Bot wizard with Hook steps 1–2 already credited

Phase 1 — Hook + Curriculum foundation

- Rewrite `journeyService.js` with 7-step spec
- Build Demo Table macro (§5.1)
- Build public Spar endpoint (§5.2)
- Build Quick Bot wizard (§5.3)
- Rewrite `JourneyCard` to support phases
- Wipe existing `journeyProgress` on deploy
- Smoke test: a new signup can complete all 7 steps end-to-end in under an hour

Phase 2 — Specialize phase

- Build `recommendationService.js` + `userActivitySummary.js` + `featureCatalog.js`
- Wire `GET /api/v1/guide/recommendations`
- Extend `JourneyCard` to render Specialize card stack
- Dismissal + 7-day suppression
- Smoke test: a user who just graduated Curriculum sees 3 relevant cards

Phase 3 — Re-engagement & polish

- 14-day inactivity nudge via socket + GuideOrb pulse
- Slide-in panel escalation (§7.4)
- Coaching card (§5.5)

- Curriculum Cup template seed + clone endpoint (§5.4)
- Rookie Cup template seed + clone endpoint (§5.8)
- Deterministic bracket seeding infrastructure (§5.9) — `seedingMode` enum, bracket generator branch, admin UI toggle

Phase 4 — Measurement

Concrete deliverables, per the §2 requirement:

- **`metricsSnapshot` table + daily cron** that computes the North Star, secondary metrics, funnel completion rates, and bucket distribution at UTC midnight
- **Admin dashboard page** (`/admin/guide-metrics`) rendering:
  - North Star (current + 30-day trend)
  - Each secondary metric (current + trend)
  - 7-step funnel visualization with drop-off per step
  - Bucket distribution pie + recommendation click-through / dismissal rates per bucket
  - Per-bucket archetype score histograms (for tuning the §6.1 normalization constants — see the discussion of "tuning blind" in §6.1)
  - Cohort slicer with admin-selectable granularity (Day / Week / Month, default Week — same query, different `DATE_TRUNC`)
- **Backfill script** for any metrics derivable from raw events (step completions, bot creations, tournament entries) so the first day of the dashboard isn't empty
- **A/B hook points** instrumented in `recommendationService` and `journeyService` so future experiments (e.g. test a new Curriculum step ordering) can be measured without re-instrumenting

12. Decisions Captured

This section records every meaningful design decision and *why* we made it, so future-us can revisit with full context.

Decision	Choice	Reasoning
Number of phases	Three (Hook, Curriculum, Specialize)	Two phases conflated low- and high-commitment moves; splitting reduces drop-off at the commitment boundary
Hook step count	2	Minimum to demonstrate "play + watch"; fewer would skip the "bots are first-class players" insight
Curriculum step count	5 (Create → Train → Spar → Compete → See Result)	Mirrors the bot lifecycle end-to-end with a satisfying payoff at step 7
Spar in the curriculum	Kept	User explicitly asked to keep it; bridges abstract training to concrete competition with low stakes
Trigger detection	Server-side for all 7 steps	Current hybrid (some client-triggered) causes state drift and testing pain
Reward structure	+20 TC at end-of-Hook, +50 TC at end-of-Curriculum — **admin-configurable**	Anchored to existing `JOURNEY_COMPLETE_TC = 50` benchmark (already in `journeyService.js`), plus a modest +20 at Hook end to introduce the credit system without overshooting. TC is not spent; with its 5× activity-score weight, +50 TC = +250 activity-score points. Values stored in SystemConfig so admins can tune without a deploy — see §8.4.

Decision	Choice	Reasoning
Number of Specialize buckets	Four (Designer, Trainer, Competitor, Explorer)	First three cover the producer lifecycle; Explorer covers legitimate consumer-only users. Folding Explorer into Competitor would undersell users who engage by watching rather than producing.
Archetype scoring	Rule-based, no ML	Pre-launch with no data; ML scoring is v2 once we observe patterns
Fresh-graduate starting state	Balanced producer — 1 Designer + 1 Trainer + 1 Competitor card, Explorer dormant	Every Curriculum graduate has exactly one action per producer bucket; Competitor-only bias would be redundant right after they finish their first tournament. Balanced surface lets the user's first pick become the first real archetype signal. Explorer is emergent (zero signal at graduation) and enters the mix only after consumer activity.
Measurement is v1, not v2	Dashboard + daily time-series snapshots ship in Phase 4 of v1	Without a view into metrics we can't tell whether the Guide is working. Snapshotting later requires painful backfill for derived metrics like bucket distribution.
Three-tier card visual hierarchy	Hero (journey) → Recommendation card (Specialize) → Tile (post-journey shortcuts)	Visual weight decreases as user familiarity grows. Large cards demand attention during learning; small tiles are efficient during mastery. Maps directly to the user's progression through the platform.
Curriculum view: hero + checklist (not single card)	All 5 Curriculum steps visible as a checklist; current step rendered as a hero card	Hook is one-frictionless-action-at-a-time. Curriculum is a multi-step commitment where seeing the ladder ahead reduces uncertainty. Checklist gives progress + preview without fragmenting focus.
SlotGrid locked during journey, unlocked at graduation	Fully hidden during Hook and Curriculum; appears at step 7 completion with auto-populated defaults and full edit access	Disabled tabs teach "I can't do this yet" — noise without value. Hiding is simpler and makes graduation feel rewarding. Restart-onboarding does NOT re-lock (once earned, always available).
Curriculum Cup: template + clone, not on-demand creation	Master template seeded once; each user's Cup is a clone with themed bot-name overrides	Faster spawn, consistent structure, admin-tunable without code. Reuses existing tournament-template-clone infrastructure rather than building a new creation path.
No Sterling in first tournament	4-entrant field: 2 Rusty + 1 Copper only. Sterling reserved for the second tournament via Specialize (8-entrant, Sterling seeded to opposite bracket arm).	First-ever tournament teaches the *flow*; losing round 1 to Sterling is a bad first experience. Progressive difficulty arcs (easy → boss) feel better than front-loaded ones.

Decision	Choice	Reasoning
Rate limits replaced with GC + retention + correctness guards	Demo tables: GC sweep (1-active/user, 2-min-post-complete, 1-hour-TTL). Spar: no rate limit, 30-day retention, 1-active-per-bot semantic guard. Curriculum Cup: 30-day retention via sweep.	Rate limits punish engagement. The real concerns (clutter, server capacity, stuck users) are solved by cleanup + retention + correctness, not capping. Progression forcing lives in recommendation logic, not endpoints.
Three new schema flags: isDemo / isSpar / isCup	Each marks a distinct privacy-scoped, auto-GC'd, ELO-excluded surface	Single-flag-per-purpose keeps semantics crisp — `isTest` (e2e hidden) stays separate. Migration is additive, existing data untouched.
Rookie Cup as a separate template (not a parameterized Curriculum Cup)	Second template alongside Curriculum Cup, targeted by a dedicated clone endpoint	Templates are blueprints; parameterizing them with "which composition" blurs the concept. Separate templates keep each Cup tier crisp, let us add a "Veteran Cup" later without disruption, and reuse the existing template-clone machinery cleanly.
Deterministic bracket seeding as general infrastructure	New `seedingMode` enum on tournaments/templates; `slotIndex`-respecting bracket generator branch	Rookie Cup's Sterling placement needs it, but building it as a Guide-specific hack is wrong. Made general so future hand-seeded tournaments (Championships, admin-curated fields, test fixtures) reuse the same foundation. Default `'random'` preserves existing behavior.
Quick Bot default: minimax novice → intermediate after first training	Uses the existing built-in minimax tiers (Rusty/Copper behaviors) rather than inventing a new default	Leverages code that already exists; the "novice starts random, intermediate after training" arc makes step 4's training *visibly change* bot behavior — pedagogically the best configuration. Real ML (Q-learning, policy gradient, etc.) is reserved for Specialize users who opt into non-minimax algorithms (discovery-rewarded via §5.7).
Coaching card: 4-branch decision tree, Curriculum-only	Champion → Rookie Cup CTA; Runner-up → Train again; 1st-round loss w/ 1 training → Train again; 1st-round loss w/ 2+ trainings → Switch algorithm. Rules in a JS config file, not SystemConfig.	Four branches is the minimum that maps to distinct next actions. The critical split (3 vs 4) prevents telling a heavy-training user to "train more" after a loss — that's a retention-killer. Curriculum-only because coaching is a teaching affordance; Specialize users have graduated into the recommendation-stack paradigm and don't need dual guidance channels.
Archetype threshold tuning: commit to defaults now, expect to tune post-launch	Five constants (D=3, T=10, C=3, F=5, E=0.1) defined with explicit rationale; admin-tunable via SystemConfig; dashboard surfaces per-bucket score histograms (not just dominant assignments)	Pre-launch we can't know the right values — but we can set up the infrastructure so finding out is quick. Admin-tunable turns tuning into a config change. Histograms give the signal we need to know when a constant is miscalibrated. Instrumented tuning beats guessing.

Decision	Choice	Reasoning
Stagnation handling distinguishes rejection from exhaustion	Decay mode (30-day dismissal streak → 1 card, no pulse) and wildcard mode (catalog exhausted → cross-bucket suggestion, weekly rotation) are separate responses	A user who dismisses every card has a different problem than one who has completed every available card. Treating them identically wastes both signals. "Just keep trying" v1 default would leave a meaningful cohort staring at stale recommendations forever — actively bad UX.
Demo table pairings: curated-random allowlist	Four pre-verified bot pairings (Copper/Sterling, Rusty/Copper, Copper/Copper, Sterling/Sterling); random selection from this list	Neither single-static (boring on repeat) nor uncurated-random (risk of broken bot surfacing). Barely more work than static and variety ships on day one. PR-editable, not SystemConfig — quality decisions shouldn't be admin knobs.
Admin experience: opt-in player Guide + role-gated tiles	Settings toggle defaults off for admins; tile catalog gets `requiredRole` metadata for admin-only tiles; admins get 2 supplemental admin tiles at graduation	Operational admins don't want to be nagged through Curriculum. Admins who dogfood can opt in. Admin tiles give fast access to admin pages without inventing a new recommendation surface. Tiles filter in picker (non-admins don't see them) and at render (defense against mis-pinned tiles).
Internal-usage pollution prevention via `isTestUser` + uniform metrics filter	New `User.isTestUser` flag defaulted true for admins / seed accounts / internal email domains; all metrics queries filter `WHERE isTestUser = false`; admin opt-in to count	Dashboards are meaningless if admin dogfooding, QA runs, and dev smoke tests count as "real users." Role-based exclusion alone misses QA non-admins; email-domain alone misses contractors. Explicit flag + uniform query-layer filter is the single source of truth. User experience stays identical — internal users still earn TC and progress; only aggregates exclude them.
`metricsSnapshot` retention: keep forever by default	SystemConfig key `guide.metricsSnapshot.retentionDays` `= null` (indefinite); optional trim if integer is set	Aggregates are ~150 KB/day, so 10 years ≈ 550 MB — storage is a non-issue. Opportunity cost of losing year-over-year trend data is much higher. Tunable safety valve exists if future-us disagrees.
`um` CLI enhanced, not replaced	Additive: new `um specialize`, `um rewards`, `um testuser`; existing `um journey` gets `--phase` shortcuts + deeper `--reset`; `um status` adds 5 fields	Existing `um` is proven QA/dev tooling. Keeping it additive preserves muscle memory for QA. The new Guide model has no matching CLI without these additions — testing the phased journey, Specialize state, discovery rewards, and `isTestUser` flag manually would be painful. Each new concept in the spec has a corresponding CLI verb.

Decision	Choice	Reasoning
Phase 0 as a first-class phase (not just "onboarding work")	New §3.5 covering visitor → registered user: live-demo hero, progressive CTA ladder, guest mode (localStorage), guest-to-user Hook credit on signup, deferred email verification	Visitor → signup is the highest-leverage conversion point on the platform and the input to everything else we've designed. The current popup-during-play pattern interrupts flow at the worst moment; guest mode + contextual ask converts at 2–5× the rate industry-wide. Landing directly in Curriculum step 3 (Hook pre-credited) rewards the visitor for their pre-signup engagement instead of making them redo it.
Guest identity via localStorage, not DB	No ephemeral-user rows, no cookies requiring consent, no privacy banner needed	Works for the 90% case (same device, same browser). Zero DB changes, zero privacy complications. Upgrade to server-tracked identity in v2 only if multi-device guest traffic is meaningful.
Deferred email verification	Post-signup user lands directly in Curriculum step 3; soft banner nudges verification; only tournament entry is gated behind verified email	Current "verify first, use second" flow kills momentum — verification emails in spam mean user drop-off. Moving the gate to tournament entry means they verify when they have a concrete reason, preserving post-signup momentum. Spam protection (honeypot, timing, OAuth) remains unchanged.
Discovery rewards — subset, not all six proposed	4 one-shot rewards: +10 universal (first Specialize action), +25 Competitor (first real tournament win), +10 Trainer (first non-default algorithm), +10 Designer (first template clone)	Three archetype activations + one universal welcome. Dropped "first recurring tournament" (too niche) and "first follow/watchlist" (+5 for following feels like TC bait). Explorer intentionally unrewarded so it emerges organically rather than via inducement. All admin-configurable.
Cross-bucket disqualifiers	Yes (§6.2–6.4 global disqualifiers)	Avoids dark patterns like "pile on more training" when the root issue is no competition
Card surfacing	Max 3 cards, 2 from dominant + 1 secondary	Balances focus with variety; prevents monotony
Dismissal model	7-day suppress, not permanent	Users change their minds; a permanent hide forecloses that
Re-engagement nudge	Socket + orb pulse, no modals	Respects user agency; modals reserved for rewards
Popups	Only for rewards	Popups for recommendations have 1–3% CTR and feel disrespectful
Journey migration	Wipe on deploy	Pre-launch; no real data to preserve
Client-triggered steps	Removed	Server-detection is cleaner; if a milestone isn't server-detectable, it's not a milestone
Programming scope	Build what the journey needs	Journey is critical for engagement; code cost is not the constraint

13. Open Questions — For Implementation Review

These are questions we chose not to resolve at requirements time but must be confirmed before or during implementation. Grouped by subject area for the next round of discussion.

13.1 Rewards & economy

1. ~~**Reward economics.**~~ **RESOLVED** — +20 TC at Hook end, +50 TC at Curriculum end. TC is an earned activity-score metric (not spent); with 5× weight in tier progression, +50 TC = +250 activity-score points. Both values **admin-configurable** via `guide.rewards.hookComplete` and `guide.rewards.curriculumComplete` SystemConfig keys (§8.4). Anchors to existing `JOURNEY_COMPLETE_TC = 50` benchmark.
2. ~~**Bonus reward for Curriculum Cup wins?**~~ **RESOLVED** — no win bonus. The Cup's composition (2 Rusty + 1 Copper) is deliberately beatable, so "winning" isn't a meaningful skill test; a bonus here would dilute the meaningfulness of later tournament wins where victory is actually earned. Victory rewards stay reserved for Specialize-phase competitive play.
3. ~~**Specialize milestone rewards.**~~ **RESOLVED** — yes, a curated four-item subset of one-shot "discovery rewards" (§5.7), each granting +10 to +25 TC the *first time* the user performs a meaningful archetype-activating action. Admin-configurable via `guide.rewards.discovery.*` SystemConfig keys. Explorer intentionally has no discovery reward — it should emerge organically from natural consumer behavior rather than TC bait.

13.2 Curriculum Cup specifics

4. ~~**Themed name pool curation.**~~ **RESOLVED** — hardcoded in `tournament/src/config/curriculumNamePools.js` with a curated initial set of 10 Rusty / 8 Copper / 6 Sterling names (see §5.4). Editing is a source-controlled PR; no live admin tuning needed since names don't require A/B experimentation or per-environment adjustment.
5. ~~**Curriculum Cup opponent scaling.**~~ **RESOLVED** — no scaling in v1. Curriculum Cup composition is fixed (2 Rusty + 1 Copper) for every user. Rationales: consistency > adaptivity for a *teaching* tournament; makes funnel metrics (Cup win-rate) meaningful; the "over-trained Curriculum user" scenario is hypothetical (only 1 training run required to complete step 4). Dynamic difficulty is a Specialize-phase concern, not Curriculum. Revisit post-launch if >50% of users sweep 3–0.
6. ~~**Second-tournament format.**~~ **RESOLVED** — separate "Rookie Cup" template (§5.8), cloned on demand via `POST /api/v1/tournaments/rookie-cup/clone`, triggered by Specialize Competitor recommendation #1. 8 slots (4 Rusty + 2 Copper + 1 Sterling), reuses the shared §5.4 clone+GC+name-pool machinery. Flag renamed from `isCurriculum` to `isCup` to accommodate both tiers.
7. ~~**Sterling seeding.**~~ **RESOLVED** — current tournament infrastructure does not support deterministic seeding, so we're adding it as new general infrastructure (§5.9). New `seedingMode` enum (`'random'` default, `'deterministic'` new) honored by the bracket generator when set on a template. Rookie Cup template sets `seedingMode = 'deterministic'` with Sterling at slot 8 (opposite bracket arm from user's slot 1). Usable by future tournaments beyond Rookie Cup.

13.3 Specialize phase

8. ~~**Quick Bot default algorithm.**~~ **RESOLVED** — minimax with starting tier `novice` (Rusty-equivalent: random moves), bumping to `intermediate` (Copper-equivalent: blocks/wins) on first training run. Reuses existing built-in minimax tiers; pedagogically justifies the first training run by making it visibly change bot behavior. Both tiers admin-configurable via `guide.quickBot.defaultTier` and `guide.quickBot.firstTrainingTier` SystemConfig keys (§8.4). Real ML happens in Specialize when a user switches to a non-minimax algorithm (rewarded via §5.7).
9. ~~**Coaching card rules.**~~ **RESOLVED** — 4-branch decision tree in §5.5 (champion / runner-up / 1st-round-loss-with-1-training / 1st-round-loss-with-2+-trainings). Rules live in `backend/src/config/coachingCardRules.js` as a plain JS array with predicate functions; editing is a PR, not a SystemConfig change. Curriculum-only scope — Specialize phase uses the §6 recommendation stack, not coaching cards. Placeholder copy; final strings are a brand-voice decision at implementation time.
10. ~~**Archetype threshold tuning.**~~ **RESOLVED** — pre-launch, we commit to the five constants (D=3, T=10, C=3, F=5, E=0.1) with explicit rationale for each (§6.1). All five are admin-tunable via SystemConfig (§8.4); the §2 dashboard surfaces per-bucket score histograms so we know *when* to tune. Post-launch tuning within the first 90 days is expected — it's the plan, not a failure.
11. ~~**Specialize card stagnation.**~~ **RESOLVED** — two distinct stagnation modes handled in v1 (§7.4). Mode A (dismissal-only streak for 30+ days): Guide decays to 1 quiet card, no pulse, reverts on any positive action. Mode B (catalog exhausted): wildcard recommendation from a non-dominant bucket, rotated weekly. Three admin-tunable SystemConfig keys control the thresholds. Distinguishing rejection from exhaustion prevents both failure modes the v1 "keep trying" default would create.
12. ~~**Archetype-seeded default SlotGrid tiles.**~~ **RESOLVED as deferred with trigger criteria** — v1 ships with static `postJourneySlots` for all graduates (§9.3). Revisit as a v2 feature **if and only if** post-launch data shows (a) ≥ 60% of graduates modify their default tiles within 7 days AND (b) the modifications cluster by archetype (e.g. Designer-dominant users disproportionately add "Create a bot"). If neither condition is met at 30 days post-launch, the feature stays deferred

indefinitely — the static default is good enough. Measurement is a small extension of the existing §2 dashboard (track SlotGrid edit events).

13.4 Feature details

13. ~~**Demo Table bot selection.**~~ **RESOLVED** — curated-random allowlist (§5.1). Four pre-verified matchups live in `tournament/src/config/demoTableMatchups.js`; random pick on each demo request. Variety ships on day one, broken-bot risk eliminated. PR-editable.
14. ~~**Admin experience.**~~ **RESOLVED** — admin experience fully specified in §9.5. (a) Player Guide is opt-in via a settings toggle (defaults off for admins, on for regular users); (b) `slotActions.js` gains a `requiredRole` metadata field so admin tiles are filterable at both picker and render; (c) admin users graduate with 2 supplemental admin tiles pre-pinned. Admin-specific recommendation stack deferred — tiles are enough for v1.
15. ~~**Measurement dashboard cohort definition.**~~ **RESOLVED** — cohort slicer supports admin-selectable granularity (Day / Week / Month, default Week). UI-level view pivot, same query with different `DATE_TRUNC` — no separate aggregation code per granularity. Admins switch based on observed signup volume (daily for high volume, monthly when buckets need to accumulate users). Zero pre-launch guessing required (§2, §11 Phase 4).
16. ~~**`metricsSnapshot` retention.**~~ **RESOLVED** — keep forever. Aggregates are tiny (~150 KB/day, ~550 MB after 10 years — storage cost is cents, opportunity cost of losing trend data is much higher). `guide.metricsSnapshot.retentionDays` SystemConfig key defaults to `null` (indefinite); set to an integer to enable trimming if future-us ever wants the safety valve.
17. ~~**Internal-usage pollution of metrics.**~~ **RESOLVED** — new `User.isTestUser` flag (§8.4) defaulted `true` for admins, seed-script accounts, and internal email domains (via `metrics.internalEmailDomains` SystemConfig). All aggregation queries filter `WHERE user.isTestUser = false`. Admin opt-in toggle in Settings for admins who want their dogfooding counted. See §2 "Preventing internal-usage pollution" for the five-layer approach.

13.5 Resolved during requirements (for the record)

- ~~Spar rate limit (20/day)~~ — resolved: no rate limit, 30-day retention + one-active-per-bot semantic guard instead.
- ~~Demo Table rate limit (5/day)~~ — resolved: no rate limit, GC sweep (1 active per user, 2-min-post-complete, 1-hour TTL) instead.
- ~~Cold-start bias~~ — resolved: balanced producer (1 Designer + 1 Trainer + 1 Competitor), Explorer dormant.
- ~~Fourth Specialize bucket (Explorer)~~ — resolved: promoted into v1 as a first-class bucket.
- ~~Curriculum Cup creation model~~ — resolved: template + clone, not on-demand generation.
- ~~Reward economics (journey TC magnitudes)~~ — resolved: +20 Hook / +50 Curriculum, admin-configurable via SystemConfig.
- ~~Curriculum Cup win bonus~~ — resolved: no bonus. Cup is training wheels; reserve victory rewards for Specialize-phase competitive play.
- ~~Specialize milestone rewards~~ — resolved: 4-item discovery-reward subset (§5.7), one-shot per user, admin-configurable via `guide.rewards.discovery.*`. Explorer intentionally excluded to let the archetype emerge organically.
- ~~Themed name pool curation~~ — resolved: hardcoded in `curriculumNamePools.js` with 10 Rusty / 8 Copper / 6 Sterling curated names. Not admin-configurable — editing is a PR.
- ~~Curriculum Cup opponent scaling~~ — resolved: no scaling in v1. Consistency > adaptivity for a teaching tournament; revisit post-launch if sweep rates are high.
- ~~Second-tournament format~~ — resolved: Rookie Cup as separate template (§5.8), cloned via dedicated endpoint, targeted by Specialize Competitor recommendation #1.
- ~~Sterling seeding~~ — resolved: adding new `seedingMode` enum + deterministic-seeding branch in the bracket generator (§5.9). General infrastructure, not a Guide-specific hack.
- ~~Quick Bot default algorithm~~ — resolved: minimax at `novice` tier (random), bumping to `intermediate` (blocks/wins) on first training. Reuses existing system-bot tier definitions; both admin-configurable.
- ~~Coaching card rules~~ — resolved: 4-branch decision tree in `coachingCardRules.js`, Curriculum-only scope. Specialize tournaments use the normal recommendation stack.
- ~~Archetype threshold tuning~~ — resolved structurally: 5 constants committed with rationale, admin-tunable, score histograms on the dashboard. Post-launch 90-day tuning expected.
- ~~Specialize card stagnation~~ — resolved: §7.4 decay mode (dismissal streak) + wildcard mode (catalog exhaustion). Three tunable knobs. Prevents the "stale cards forever" failure mode the original "keep trying" default would have shipped.
- ~~Archetype-seeded default SlotGrid tiles~~ — resolved as deferred with trigger: v1 ships static defaults. Revisit as v2 only if post-launch data shows ≥ 60% of graduates customize within 7 days AND edits cluster by archetype. Otherwise

permanently deferred.
- ~~Demo Table bot selection~~ — resolved: curated-random allowlist of 4 pairings, PR-editable. Not admin-configurable — quality decisions shouldn't be knobs.
- ~~Admin experience~~ — resolved (§9.5): opt-in player Guide for admins + role-gated SlotGrid tiles via new `requiredRole` metadata + 2 admin tiles auto-pinned at graduation. Admin recommendation stack deferred.
- ~~Measurement dashboard cohort granularity~~ — resolved: admin-selectable at view time (Day/Week/Month), default Week. No pre-launch guess needed.
- ~~`metricsSnapshot` retention~~ — resolved: keep forever (default). Tunable via SystemConfig if future-us changes mind.
- ~~Internal-usage pollution~~ — resolved: `User.isTestUser` flag + uniform metrics filter. Admin opt-in to count.

14. Success Criteria

Phase 0 is successful when:

- Landing → signup conversion rate is **2×** the pre-redesign baseline (measured once we have 30 days of comparable traffic)
- ≥ 60% of new signups arrive with at least one Hook step already credited from guest mode (signal that the guest-to-user transfer is working)
- Median time from first landing visit to signup is **under 10 minutes** for users who do sign up
- No measurable increase in spam-account creation (honeypot/timing metrics stable post-launch)

Phase 1 is successful when:

- 80% of new signups complete Hook steps 1 and 2 within their first session
- 50% of new signups complete Curriculum step 3 (first bot created) within 24 hours
- 30% of new signups complete Curriculum step 7 (first tournament result) within 7 days

Phase 2 is successful when:

- ≥ 40% of Specialize-phase users click a recommendation within their first session in Specialize
- ≥ 60% of recommendations are not dismissed (they either get acted on, or stay there as "I'll get to it")

Phase 3 is successful when:

- Re-engagement nudges increase 14-day retention by ≥ 5 percentage points vs. a no-nudge control cohort
- Coaching cards surface on 100% of Curriculum step 7 completions without crashing

Phase 4 is successful when:

- We can state, in one sentence per metric, how the Guide is performing at any moment by looking at the dashboard

---

Appendix A — Glossary

- **TC** — Tournament Credits, the in-platform currency for tournament entries. Awarded at journey milestones and through gameplay.
- **Journey** — the ordered sequence of Hook + Curriculum steps a new user progresses through.
- **Phase** — one of {Hook, Curriculum, Specialize}. Every user is in exactly one phase at any moment.
- **Bucket / Archetype** — one of {Designer, Trainer, Competitor}. Every user has a score in each; bucket assignment is dynamic based on activity.
- **Card** — a single Specialize-phase recommendation rendered in the Guide. Up to 3 at a time.
- **Spar** — a casual, non-tournament bot match, typically user's bot vs. a system bot.

Appendix B — File Touch List

New files: - `backend/src/services/recommendationService.js` - `backend/src/services/userActivitySummary.js` - `backend/src/config/featureCatalog.js` - `backend/src/routes/tablesDemo.js` (if not folding into `tables.js`) - `backend/src/routes/__tests__/recommendationService.test.js` - `backend/src/routes/__tests__/tablesDemo.test.js` - `backend/src/routes/__tests__/spar.test.js`

Modified: - `backend/src/services/journeyService.js` - `backend/src/routes/guide.js` - `backend/src/routes/bots.js` (Quick Bot endpoint) - `backend/src/routes/botGames.js` (practice endpoint) - `backend/src/routes/tables.js` (demo endpoint, if not new file) - `landing/src/components/guide/JourneyCard.jsx` - `landing/src/components/guide/GuideOrb.jsx` - `landing/src/store/guideStore.js`

Removed behavior: - `POST /api/v1/guide/journey/step` (all steps now server-detected) - Client-side step 2, 4, 7 trigger code